NEA - Group Creator for School Projects

Amelie Machin

Contents

1 Analysis **2**
1.1 Problem statement . . . 2
1.2 Research . . . 2
1.2.1 Sources of inspiration . . . 2
1.2.2 Algorithm . . . 3
1.2.3 Inputting data . . . 4
1.2.4 Accounts . . . 4
1.2.5 Internal data storage . . . 5
1.2.6 External data storage . . . 5
1.2.7 GUI library . . . 5
1.3 Dialogue . . . 6
1.3.1 Initial interview . . . 6
1.3.2 Follow-up questions . . . 7
1.4 Objectives . . . 8
1.5 Modelling . . . 14

2 Design **17**
2.1 GUI . . . 17
2.2 Databases . . . 24
2.3 Inputs and outputs . . . 24
2.4 Algorithm . . . 26
2.4.1 Pseudocode . . . 26
2.4.2 Fitness algorithm . . . 28
2.4.3 Diagrams . . . 28

3 Technical solution **32**
3.1 main.py . . . 33
3.2 geneticAlgo.py . . . 59
3.3 CreateTables.sql . . . 69

4 Testing **71**
4.1 Video . . . 75
4.2 Testing plan . . . 75

5 Evaluation **79**
5.1 Objective evaluation 79
5.2 Third-party feedback 79
5.3 Feedback evaluation 79
5.4 Improvements 79

1 Analysis

1.1 Problem statement

To form the perfect grouping for a group project within a school, many factors need to be taken into consideration to prevent students from feeling incapable of completing the project. These factors include, but are not limited to, skills of the students, skills required by the project, and number of students in the group.

This project aims to create a tool to aid teachers in ensuring the students are separated into groups where the students are best able to get along and combine their individual skills to achieve their project's aims. The user can input the data that they want to be taken into consideration by the program and it will output a set of groups. This is in the form of a table on a GUI - to present the results in a readable way - and an option to export the data.

This program will not find, assume, or generate information about the students or project, it will only work with the data inputted to assign a group of students to a project.

Teachers will benefit from this tool as they don't have to spend as much time or effort creating groups for the projects they set for their students, so they can spend their time and effort on helping and teaching the students instead. Also, when the students are working on their projects, they are more likely to be focused on their work, so require less oversight from the teacher.

Students will also benefit from this tool as they would be able to apply their skills to a project in a group of people they can enjoy time with, rather than being stuck on a project they have no interest in with people they do not get along with. This means they are more likely to enjoy their time at school, be able to get more work done, and therefore be able to learn more, as opposed to being unmotivated and distracted in a poorly made group.

1.2 Research

1.2.1 Sources of inspiration

1. Random group generator from ClassTools.net [Fig. 1]
   https://www.classtools.net/random-group-generator/

   - This tool is used frequently by my teachers to create random groups for group activities.

- The GUI is very simple to use as there is little data that needs to be entered, so can all fit down the left of the screen.
- The output is very easy to read as each group is visually separated using both coloured boxes and numbering. [Obj. 3b]
- However, this tool is too simple because it lacks any way to group with conditions, which is part of my scope, as they are purely randomised. This means that teams could quit often be unfairly balanced in terms of skill, or group students together that are known to not work well together. The teacher's only option to solve this is to re-roll until the output is suitable enough for the task manually. This gave me the idea for the algorithm to be the simulation of this re-rolling and selection of optimal outputs.

2. keamk [Fig. 2] [Fig. 3]
   https://www.keamk.com/random-team-generator
   - This tool creates groups that all have a balanced level of skill.
   - I like how the skill level is inputted as a rating between 1-5 as this is intuitive for the user.
   - I also like the extra details that are outputted, such as how many are in each group next to the group name and the skill levels next to each individual. [Obj. 3a]
   - Furthermore, the option to go back to make small changes to the data inputted and get new groupings was very quick and useful. [Obj. 1g]
   - However, this tool can only create balanced groups, not groups with different objectives as is in my scope, and there is no way to assign a certain number of participants to each group, as the program puts the same number (or one away) of participants in each group. [Obj. 2c-d]

These tools influenced me to reduce the complexity of my GUI, and gave me ideas such as to keep groups visually separate using colour as well as tables, adding the option to go back and edit the inputted data later, and some summary statistics to include with the outputs.

1.2.2 Algorithm

I have researched into representing students and projects into bipartite graphs, and then using matching algorithms such as the alternate path algorithm to join students to projects [Fig. 4]. However, this will only match one student to one project, not groups so is not suitable. I then researched applying the algorithm repeatedly to graphs with randomised students, but this means that some students will never be joined into a group purely based on chance and that the groups formed are such that each individual is best for the project, not that the overall group is best suited for the project, so teamwork will not

as good as it could be. I then researched using a grouping algorithm, such as https://help.feedbackfruits.com/hc/en-us/articles/23527084211346-Group-Formation-Algorithm-Explainer to create similar groups then using bipartite matching algorithms to match them. However this only creates groups with similar skill sets, not skill sets that vary based on the project.

I decided on a genetic algorithm where each individual is made up of a possible solution and the fitness score is an average between how suited each group is to its assigned project [Fig. 5]. The individuals with higher fitness scores are discarded and the lower ones are randomly selected to crossover to form new individuals. Some of these individuals formed may contain error such as the same student multiple times, or a missing student, so that must be corrected before continuing. Then, random mutations are applied to some individuals, which may involve swapping of students between groups, moving some students to an optional project, or moving some students to group that isn't full. New fitness scores are calculated and the process repeats a predetermined number of times [Fig. 9]. I chose this because genetic algorithms are very adaptable to varying conditions and constraints, such as the ones that the user chooses in this program. This will allow the program to test different combinations of projects and students, test different group sizes for a project, and swap in and out optional projects to find better and better solutions until it reaches a high-quality solution that fits all the user's needs. This fits the project scope far better than all previously researched algorithms.

1.2.3 Inputting data

For the inputting of data, I have decided that it can be inputted through txt and csv files [Obj. 1a], manually on the GUI [Obj. 1b], or any combination of those methods [Obj. 1c]. This is because the data for different individuals and different projects could be stored in different ways within the same system, and manual tweaks may be needed by the user to ensure all data is as is needed by the application. I will also include the option to import the data from an Excel spreadsheet at the request of my third party [Obj. 1b].

1.2.4 Accounts

I have decided against implementing an account system because I don't think it will provide any value to the program. Accounts are typically used for ensuring data can only be accessed by the users with the correct permissions. As this program will be designed to be run locally, all the data will be stored and only accessible on the machine the program was run on, so the OS can handle user security.

Another reason for accounts is allow users to access their data that has been stored by a third party, especially within a cloud to allow for data to be quickly transferred between devices. I will implement a manual solution to this by allowing users to export and import all the data the program stores and the user can handle that data transfer themselves.

1.2.5 Internal data storage

I chose to store the data about the students and projects in their own classes, that are aggregated within an 'individual' class. The student and project classes contain their names, skills and the project will have the recommended group size. The individual will contain a project object, student objects, and current fitness score as well as methods for calculating fitness scores, crossover with other individuals, and mutations. [Fig. 7]

This keeps the attributes and methods stored together in the program to make the code easier to read and understand by a programmer.

1.2.6 External data storage

[Obj. 4] I have researched into whether information about projects and students should be stored somewhere, and if so, how. I have concluded that they should be stored in relational databases that are accessible and modifiable by entering the details into the graphical interface, which are passed as parameters into SQL statements. These can then be loaded as a list of checkboxes that can be selected and deselected by the user for that run. [Obj. 1c]

I chose to store the data because being able to make minor changes to the data and then re-running the program quickly is a very valuable feature that could save users' time and energy as they would not have to go through the entire process of entering every piece of data just for small changes to some of the data. This feature is present in keamk's grouping tool, and I found it very useful in my testing.

I chose databases to store the data because I have more experience working with relational databases and SQL then any other form of data storage, such as in files. The data is structured so databases would allow for efficient access and modification of specific entries.

1.2.7 GUI library

Between the two main python GUI libraries, pygame and Tkinter, I chose Tkinter because of the larger library of classes of pre-made, customisable widgets that can be used more easily to produce a simple, easy-to-use tool, such as a dialog that allows users to graphically navigate through their files to find their data file [Fig.6].

On the other hand, with my prior experience with pygame, I have found it to be more suited towards building games and graphically-complex tools, which is not what I am aiming for in this project. I am aiming to have a GUI that is quick and simple to create and edit, and has user-friendly features for inputting, editing, and reading data. I also have more experience with Tkinter, which will reduce the amount of time I spend on the GUI and allow me to focus on developing and improving the grouping algorithm and user experience.

1.3 Dialogue

My third party is Dr Ed Horncastle, the lead teacher for the Exeter Mathematics Certificate (EMC) at my school, which is where groups of 3 or 4 students work together on research projects set by University academics or companies. He is suitable because he has to make these groups based on the skills of the students and match them to projects every year, so he has a thorough understanding of how this tool could work and would benefit from using this tool.

1.3.1 Initial interview

The following questions were asked during the research stage:

1. What is the current process for creating EMC groups?
    - All the metrics to consider are gathered into a spreadsheet and then groups are formed by hand.
    - Some projects don't end up getting groups assigned, especially those that have been set by a company that already has another project with an assigned group.
2. What are these metrics?
    - Students' own choice
    - There must be a mix of genders
    - The projects must be relevant to the A-level studies of that student. Someone who does not take computer science shouldn't be given a computer science based project
    - Those that work hard and those that laze off more easily should be mixed together. If this is their first project, this is inferred from GCSE results, otherwise it is based off the results of previous projects.
    - Special educational needs
    - Timetabling restrictions
    - Students do not get put into groups with others they have worked on projects with before
3. How are the student choices measured?
    - For some of the projects, the students give their top 3 choices of projects. Starting with the least popular project, the groups are formed by trying to minimise the sum of the positions the student's ranked the project in.
4. How is the data about the students and projects currently stored?
    - Excel spreadsheet
5. How would you prefer to input the data?

- Student choices from a Microsoft Form and all other data from a Excel spreadsheet

6. How much data and in what format do you think the data should be outputted?

    - In tables with the project name, student names and summaries of information about each group such as gender ratio, how many take computer science or physics, and how many have special education needs.
    - As a simplified register in alphabetical order

7. How useful would storing project/student details locally, that can be loaded when needed, be for you?

    - Very

8. Are there any other features that would be useful for this tool to include?

    - Some extra features that could be added is timetabling for what order the projects are presented in, and itinerary for group trips out of school.

After this, I decided to include Excel spreadsheets as an input method [Obj. 1b], include options for projects to not have groups assigned despite being a part of the algorithm [Obj. 1f], take genders into account [Obj. 2e], have summary statistics presented alongside the groups [Obj. 3a], have the option to view the groups without any extra information [Obj. 3b], store the data locally for re-use at another time [Obj. 4].

I decided not to include student choice, student personality, special education needs, timetabling restrictions, or preventing grouping the same students together across multiple as adding too many restrictions to the algorithm will make it take it longer to run to create a set of groups that best matches all of these restrictions. These are the restrictions that were lower on Dr Horncastle's priorities. I will also not include timetabling or scheduling as these deviate quite far from my project's scope.

1.3.2 Follow-up questions

The following questions were asked during the designing stage:

1. I have decided on using three factors to determine how "good" a group is: the number of students with at least one relevant skill to the tasked project; the number of skills required by the project that the group meets; whether or not there are at least two different genders in the group. Could you rate how important these factors are in your opinion, or whether you think any are equal?

- The students being able to contribute is twice as important as the other two factors combined
- Project skills are slightly more important than gender

1.4 Objectives

Objectives beginning with a * are extra objectives, only implemented if time allowed and are not required for the program to meet the scope of the problem.

1. Data can be entered into the program
   (a) Data can be entered manually into the GUI
   (b) Student and project data can be entered through file import
      i. File type can be csv
      ii. *File type can be txt
      iii. *File type can be a SQLite3 database
      iv. *File type can be xlsx
   (c) Data can be loaded from stores from previous uses of the program
   (d) Data can be entered through a combination of other methods
   (e) Student data must contain a name, gender, and skills
   (f) Project data must contain a name, maximum group size, minimum group size, whether or not the project must have a group or not, and skill requirements
      i. Maximum group size must be greater than 0 and greater than minimum group size
      ii. Minimum group size must be greater than 0
   (g) Settings data must contain a target fitness, max number of iterations, and the size of the populations
      i. Target fitness but be between 0 and 1
      ii. Max iterations must be a positive integer, not including 0
      iii. Size of populations must be a positive integer greater than or equal to 4
   (h) Project and student data can be added, edited, and removed at any time before the genetic algorithm runs
   (i) At least one student, at least one project, target fitness, max iterations, and size of populations must all be entered/have values before the algorithm is run
2. Students must be grouped together and to a project
   (a) All students must be put into one group each
   (b) *All students must have at least one skill that will allow them to contribute to the project, when possible

(c) Projects must not have more students assigned to it than the maximum limit inputted by the user

(d) Projects must not have less students assigned to it than the minimum limit inputted by the user

(e) *Groups must have a mix of genders, when possible

3. The resulting groups must be viewable

(a) The user must have to option of coloured tables displayed on the GUI

i. These tables contain the project name, student names, number of students, gender ratio, and skill range

ii. Adjacent tables are coloured differently from each other

(b) The user must have the option of a register layout displayed on the GUI

i. Each project and its details are stored in separate boxes with a background distinct from its adjacent neighbours

ii. Each box contains the project name, and the names of students in that group

iii. Projects are displayed in alphabetical order

iv. The students in each group are displayed in alphabetical order

(c) Both above options can be exported to locally stored files

i. File type can be csv

ii. *File type can be txt

iii. *File type can be xlsx

4. All data must be stored

(a) The data about projects, students, and results must be stored in relational databases

(b) The user must have the option to export the databases' contents to locally stored files

i. File type can be csv

ii. *File type can be txt

iii. *File type can be xlsx

(c) *The settings data must be stored as a JSON file

5. The design must be consistent across all pages

(a) Text is black with the Arial font

(b) The top of each page that displays data must feature a header

i. The header contains text that provides a title for the data shown

ii. The title text must be displayed on the left with a significantly larger font to the data shown

iii. The header colour must be of hex code #FFDB32

iv. The header may contain buttons, solely for navigation, on the right

(c) Data about different entities must be visually separate by keeping all the data about each single entity within individual boxes

(d) Buttons that perform a function that cannot be performed at that time must be greyed out, and feature text below stating how to enable the button

(e) Buttons that lead to a different page must feature the name of the page, or the relationship between the current page and next (e.g. "back")

(f) Up to 6 colours can be used to visually separate entities through colour. Red, orange, yellow, green, blue, purple. These colours must have RGB values all greater than 186 to ensure the text remains legible

6. Users must be happy with the program

(a) 100% of users must be happy with navigating the program

(b) 100% of users must be happy with response times

(c) 100% of users must consider the groups outputted by the program to be suitable based on the provided data

(d) 100% of users must consider the methods of data entry to be convenient

(e) 100% of users must consider the methods of exporting data to be simple

(f) 100% of users must be able to read all data shown on the GUI

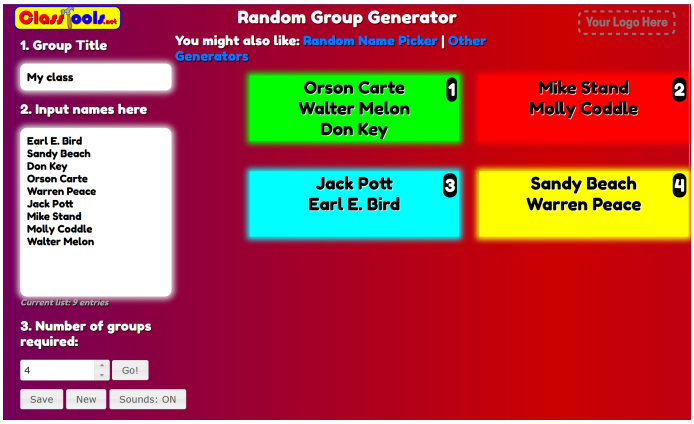


Figure 1: ClassTools's random group generator tool with inputs and outputs visible on the same page

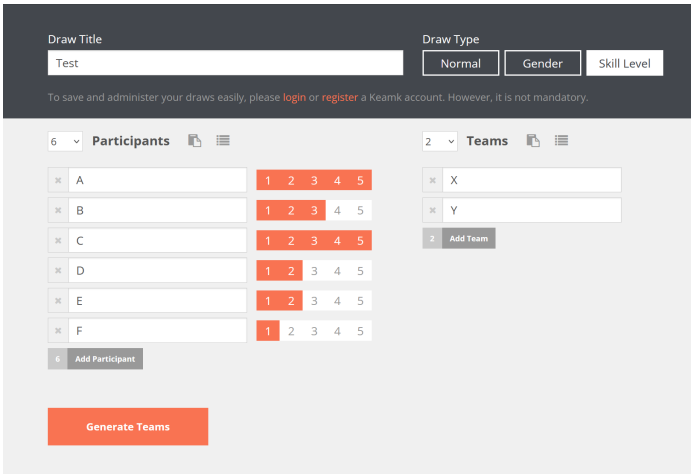


Figure 2: Keamk's menu for inputting data



Figure 3: Keamk's outputs

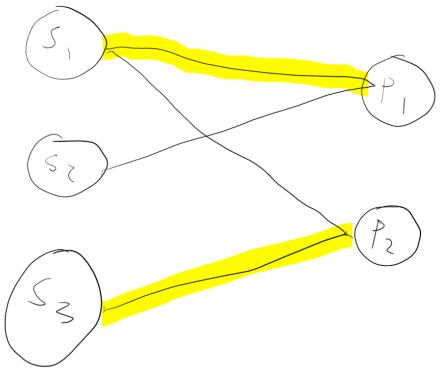


Figure 4: A bipartite graph with 3 student nodes on the left, 2 project nodes and the right, and some edges between them. A matching algorithm has been applied, with the matching highlighted in yellow. This has not met the scope because a student has not been matched.

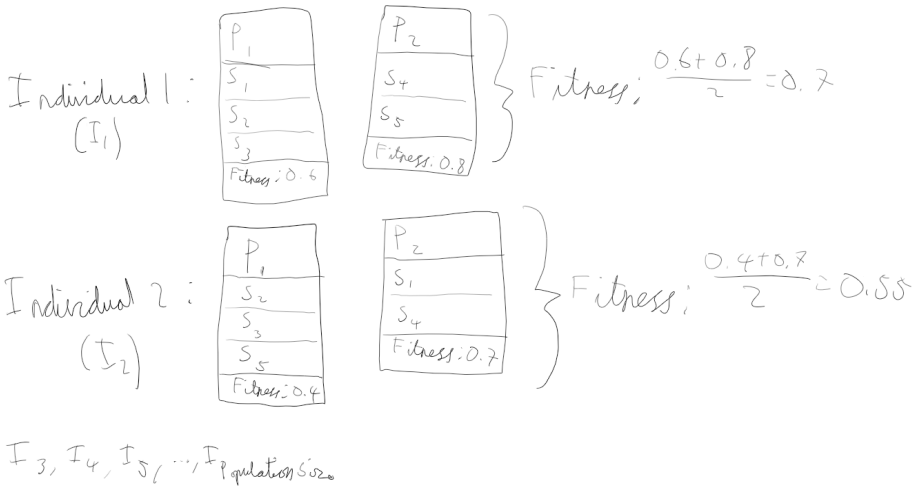


Figure 5: A sketch showing an example of a population, representing a set of possible solutions (individuals). Each individual represents a possible set of groups, and has a fitness score that represents how well every project's group matches the project's and user's requirements. This is calculated by averaging all the fitness scores of the groups it contains.

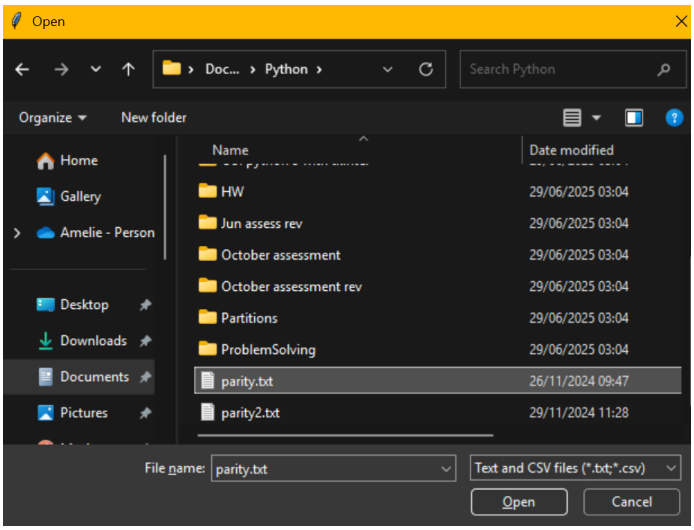


Figure 6: Tkinter's file dialog, configured to only show .txt and .csv files

1.5 Modelling

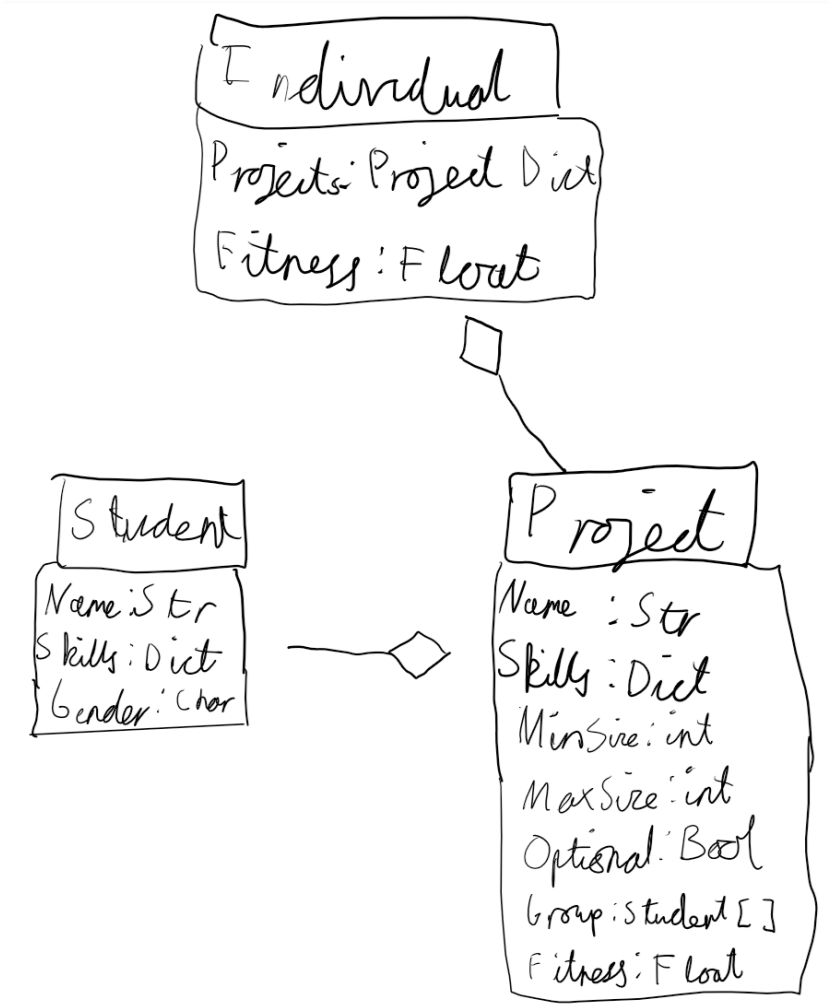


Figure 7: Class diagram showing relationships between the classes and their attributes used by the genetic algorithm.

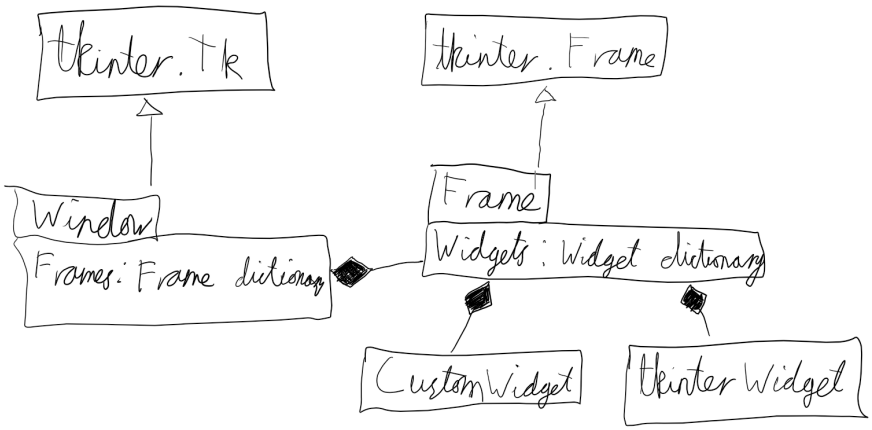


Figure 8: Class diagram showing relationships between the classes that form the GUI

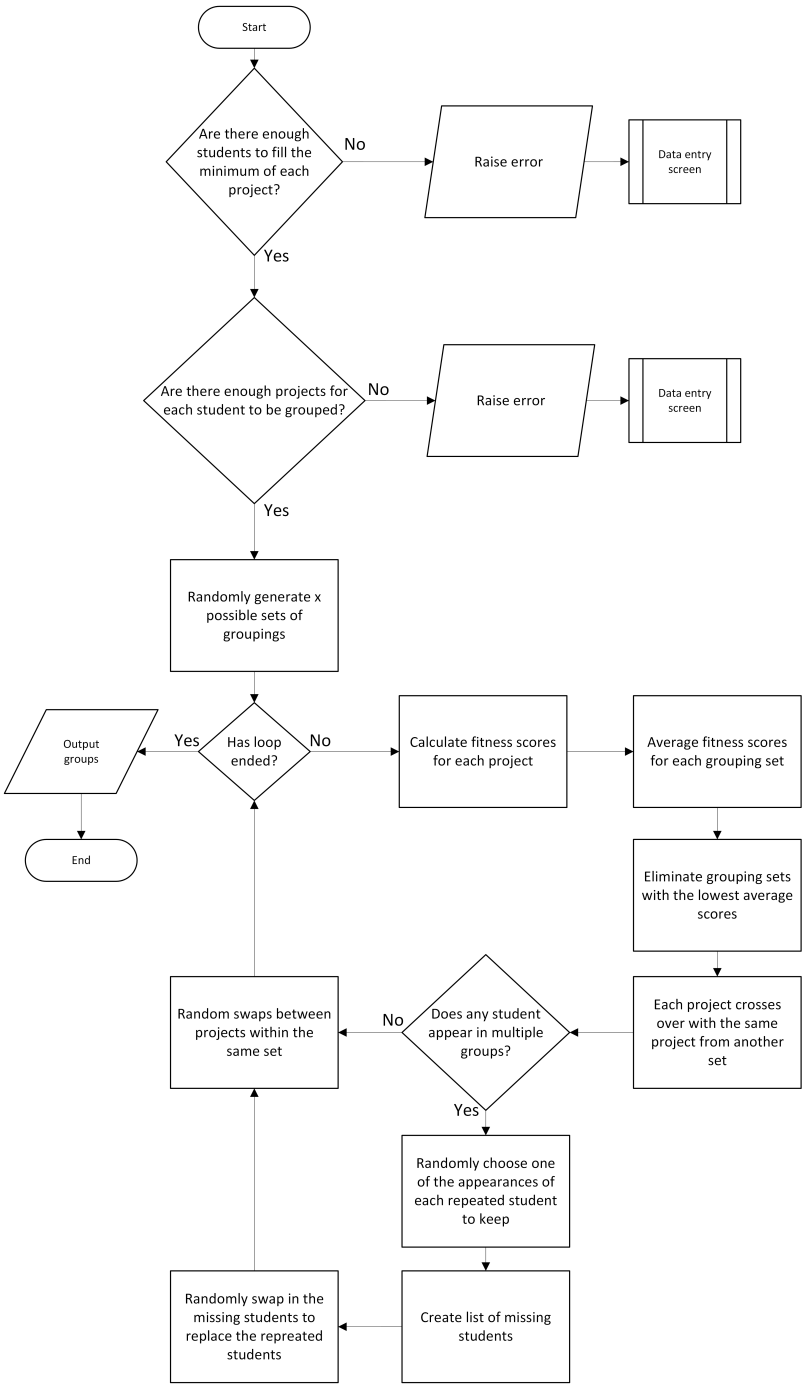


Figure 9: Flowchart showing the process for the genetic algorithm

2 Design

2.1 GUI

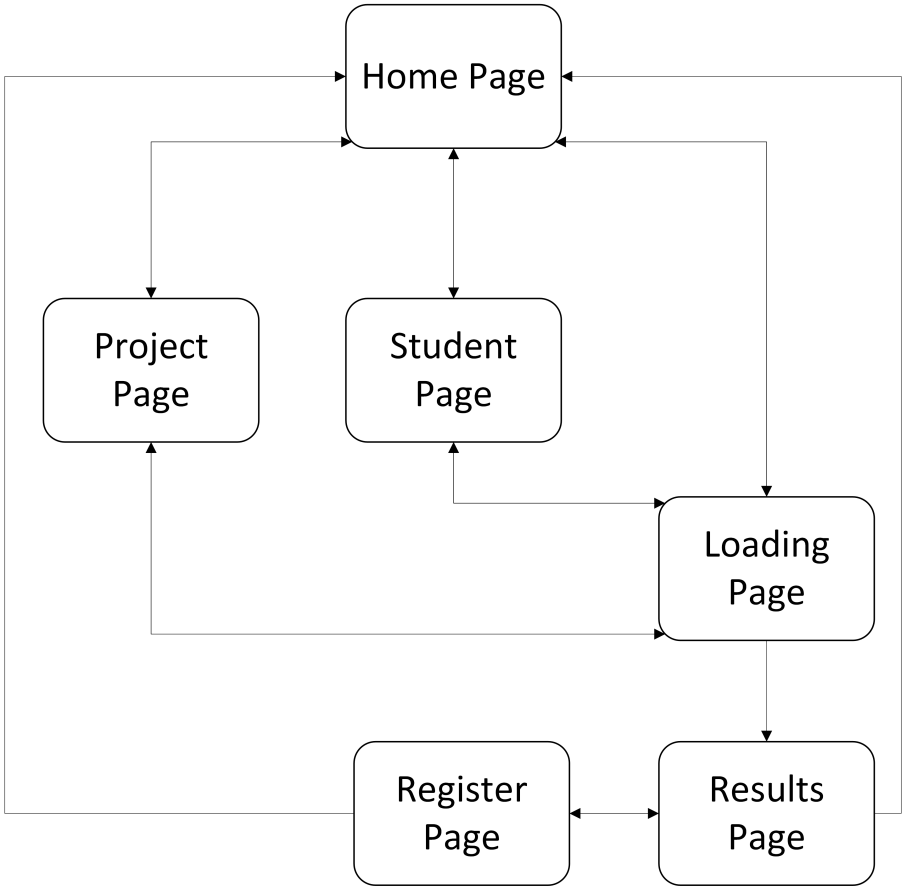


Figure 10: Page navigation diagram showing which pages can navigate to each other. Note: The student and project pages can only navigate to the previous page.

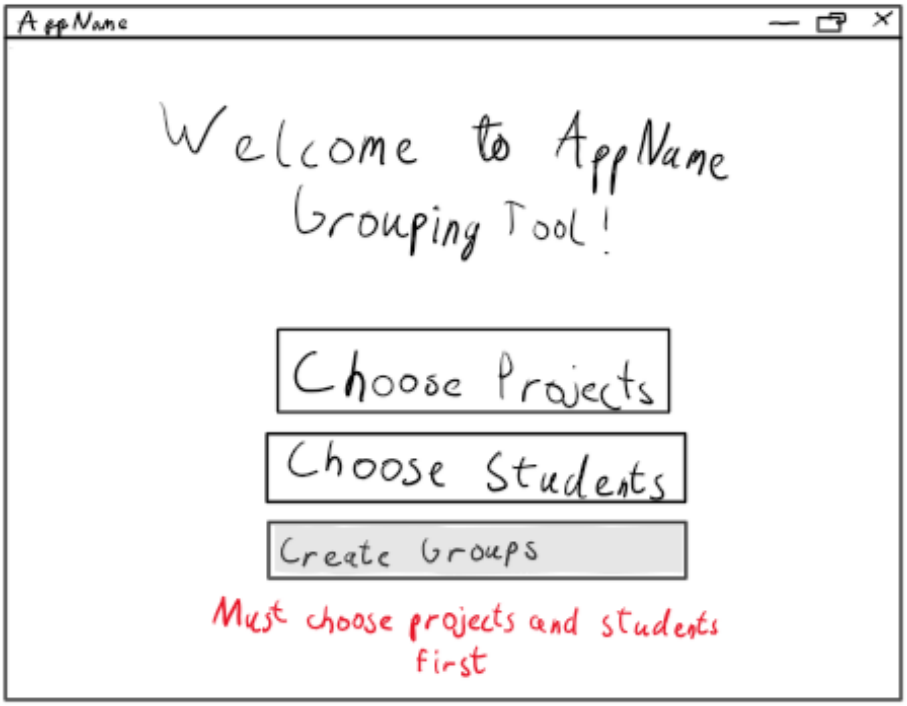


Figure 11: Sketch of the homepage layout that is the first page seen upon opening the app. The "Choose Projects" and "Choose Students" lead to the projects and students pages, respectively. The "Create Groups" button leads to the loading page and runs the grouping algorithm with the students and projects the user has selected. It is greyed out and not clickable unless at least one project and student has been selected [Obj. 1h].

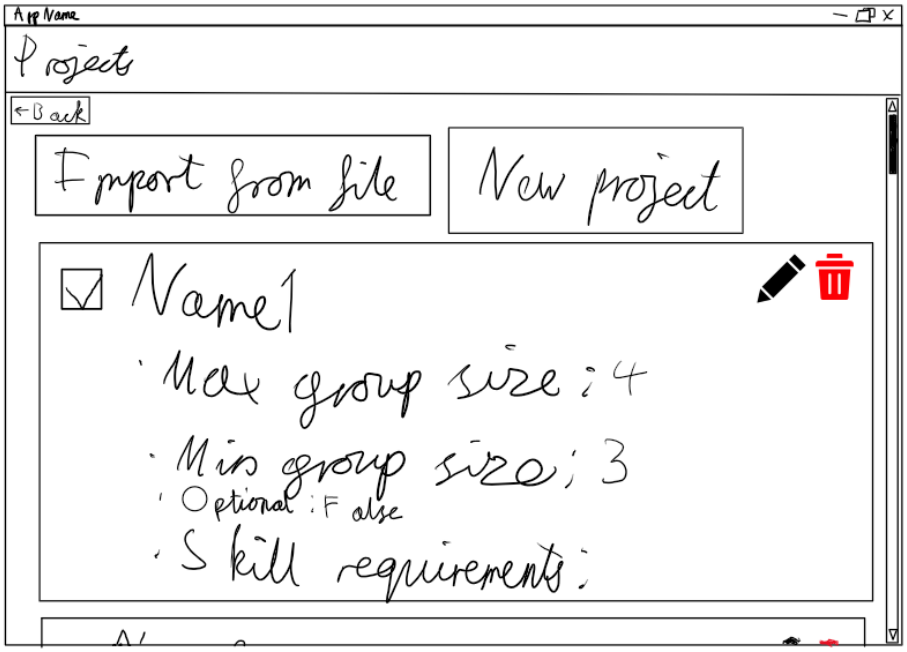


Figure 12: Sketch of the project page layout. The projects in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import project data from supported files [Obj. 1b] and to manually enter project details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The project details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each project in the list features a checkbox, allowing users to select which of the projects in the database will be used in the grouping algorithm. Projects not selected will not have groups matched to them. This page also features a back button to go back to the home page.

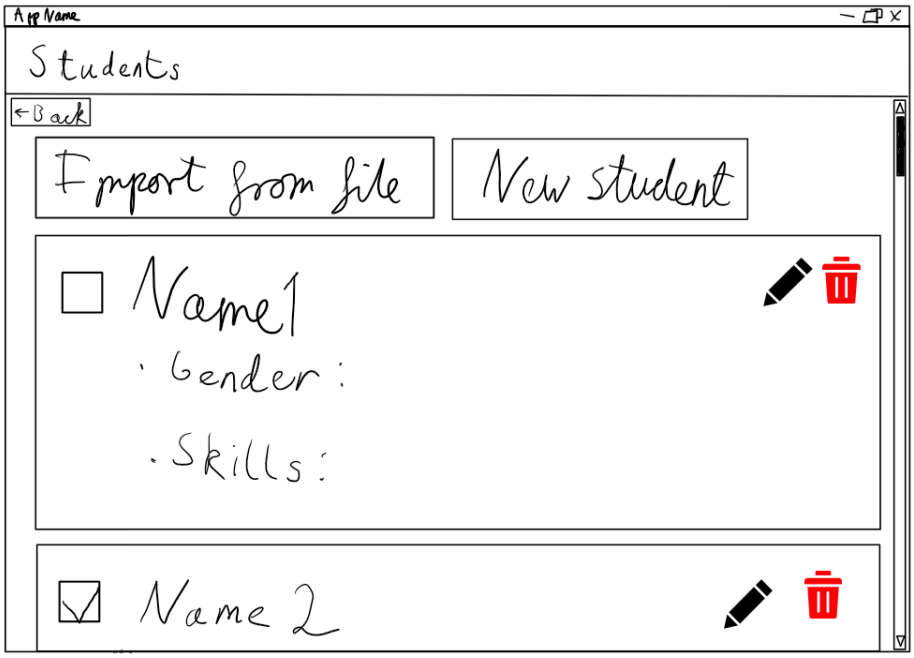


Figure 13: Sketch of the student page layout. The students in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import student data from supported files [Obj. 1b] and to manually enter student details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The student details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each student in the list features a checkbox, allowing users to select which of the students in the database will be used in the grouping algorithm. Students not selected will not be put into groups. This page also features a back button to go back to the home page.

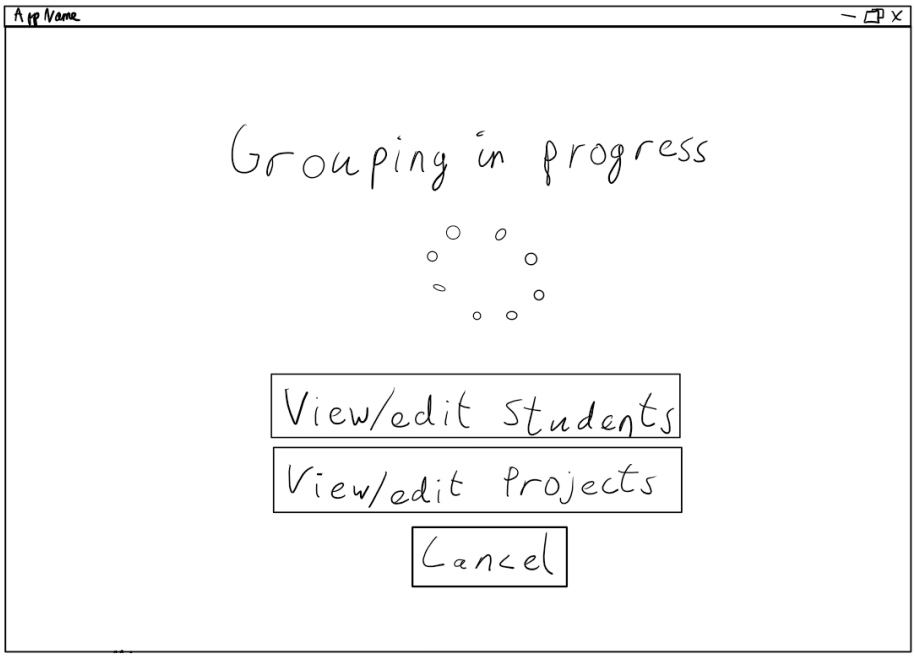


Figure 14: Sketch of the loading page layout. This page features a loading animation, and three buttons. The "View/edit Students" and "View/edit Projects" buttons lead to the Students and Projects pages, respectively [Obj. 1g]. The "Cancel" button ends the algorithm and leads to the home page. Once the algorithm has finished successfully, it moves onto the results page.

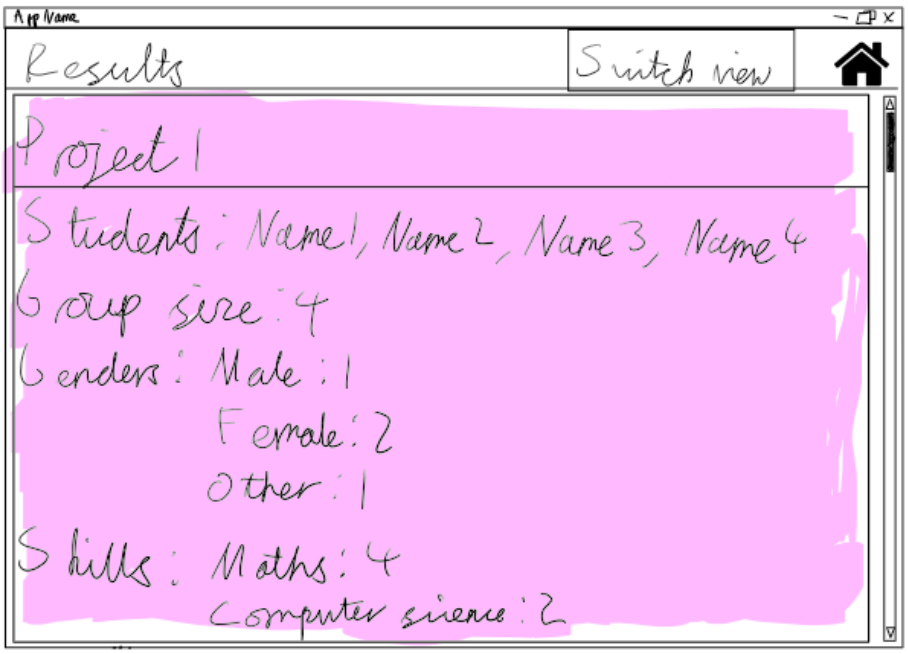


Figure 15: Sketch of the results page layout. It features a scrollable list of all the projects with the students grouped to it and summary statistics about the group, such as group size, gender ratio, and skills [Obj. 3a]. The "Switch view" leads to the register page and the home button leads to the home page.

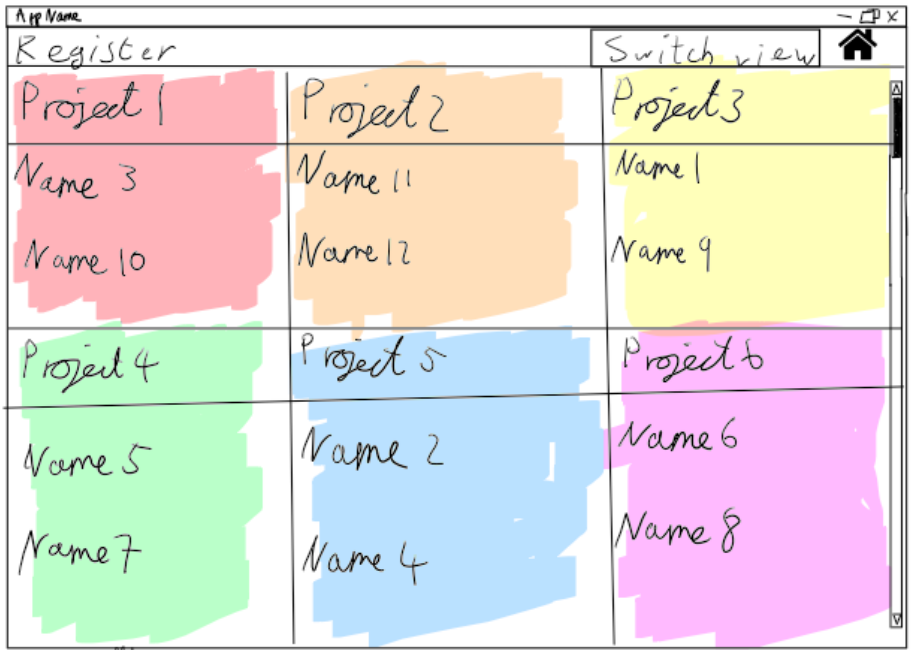


Figure 16: Sketch of the register page layout.

2.2 Databases

- Students(StudentID, Name, Gender)
- StudentSkills(StudentID, Skill)
- Projects(ProjectID, Name, MaxSize, MinSize, Optional)
- ProjectSkills(ProjectID, Skill)
- Results(ResultID, ProjectID, LocalTimeGenerated, EpochTimeGenerated)
- ResultStudentLink(ResultID, StudentID)

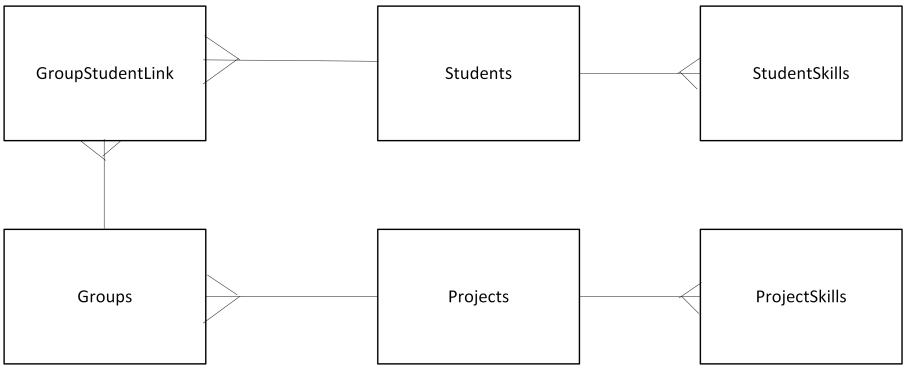


Figure 17: An entity relationship diagram showing the one-to-many relationships between the Students and Projects entities and the store of their skills to ensure all fields are atomic so the database is normalised.

2.3 Inputs and outputs

Table 1: An IPSO chart showing what data is inputted, how it is processed and stored, and how the results are outputted.

Input	Process	Storage	Output
Project details: - Name - Max group size - Min group size - Optional - Skill requirements	Validate input	Projects and ProjectSkills tables in database	Display on GUI [Fig. 12]

Student details: - Name - Gender - Skills	Validate input	Student and StudentSkills tables in database	Display on GUI [Fig. 13]
Settings: - Directory to export database to - Target fitness for genetic algorithm - Max iterations for genetic algorithm - Size of populations for genetic algorithm	Validate input	JSON file	NA
"Create Groups" button on homepage pressed	Run genetic algorithm[List. 1]	Classes for Student, Project, and Individual [Fig. 7]	Scrollable list of projects and groups [Fig. 15]

Table 2: Validations and restrictions on data entry

Data	Data Type	Validation and Restrictions
Projects		
Name	String	- Required
Max group size	Integer	- Required - Positive integers only
Min group size	Integer	- Required - Positive integers only - Must be less than or equal to 'Max group size'
Optional	Boolean	- Lack of entry is equivalent to entry of false
Skill requirements	String	- Entries separated by commas
Students		
Name	String	- Required
Gender	String	
Skills	String	- Entries separated by commas
Settings		

Directory to export database	String	- Must be a valid directory - Program must have read-/write permissions in that directory
Target fitness for genetic algorithm	Float	- Default is 0.9 - Must be between 0 and 1 inclusive
Max iterations for genetic algorithm	Integer	- Default is 50 - Must be a positive integer not including 0
Size of populations for genetic algorithm	Integer	- Default is 100 - Must be a positive integer greater than or equal to 4

2.4 Algorithm

2.4.1 Pseudocode

Listing 1: Pseudocode for the function that runs when on the loading page. It ensures it is possible to find groupings with the given requirements, then runs a genetic algorithm to find a good set of groupings. Some subroutines have flowcharts below.

```
1  TARGET_FITNESS = integer
2  MAX_ITERATIONS = integer
3  POPULATION_SIZE = integer
4
5
6  students = set of 'student' objects
7  projects = set of 'project' objects
8
9
10 #Ensure there aren't too many or too few students to meet
       ↪ requirements
11 minStudents = 0
12 maxStudents = 0
13
14 REPEAT for all projects:
15     maxStudents += project.maxSize
16     IF (not project.optional):
17         minStudents += project.minSize
18
19 #Check for too few students
20 IF (length of students < minStudents):
```

```
21      DisplayError("Error: Only {length of students} to
            ↪ fill a minimum of {minStudents} places.")
22      HomePage()
23
24  #Check for too many students
25  ELSE IF (length of students > maxStudents): #Checks for
        ↪ too many
26      DisplayError("{length of students} is too many to fit
            ↪  in {maxStudents} places.")
27      HomePage()
28
29
30  #Generate initial grouping sets and average fitness
        ↪ scores
31  population = list
32  REPEAT for i = 0 to POPULATION_SIZE - 1:
33      groups = GenInitialGroups(projects, students)
34      avgFit = (SUM fitness of each group)/(length of
            ↪ groups)
35      APPEND groups,avgFit to population
36
37  SORT population by avgFit
38
39  #Genetic algorithm
40  REPEAT for i = 1 to MAX_ITERATIONS:
41      #Ends early when one set of groupings is good enough
42      IF (highest avgFit > TARGET_FITNESS):
43          BREAK
44
45      #Eliminate the worst half of grouping sets
46      population = first (POPULATION_SIZE FLOORDIV 2) of
            ↪ population
47      new_pop = []
48
49      #Create next population
50      REPEAT for j = 1 to 90% of POPULATION_SIZE:
51          #Crossover random groupings
52          groups = Crossover(random group from population,
                ↪ random group from population, students)
53
54          #Ensure all students appear once
55          valid = verifyGroupings(groups)
56          IF (not valid):
57              fixGroupings(groups)
58
```

```
59          #Random mutations to prevent convergence towards
                ↪ a local best solution
60          Mutate(groups)
61
62          #Find the average fitness score for each new
                ↪ grouping
63          avgFit = (SUM fitness of each group)/(length of
                ↪ groups)
64          APPEND groups,avgFit to new_pop
65
66      #Combine the crossover with 10% elitism
67      population = new_pop + top 10% of population
68      SORT population by avgFit
69
70
71  return group with highest avgFit
```

2.4.2 Fitness algorithm

The fitness algorithm takes in the data about the current set of groupings and outputs a value between 0-1 that represents the suitability of the students to each project (0 being the worst possible groupings, 1 being absolutely perfect). This algorithm needs to allow the program to continually converge towards 1, guiding the program in the correct direction and helping it to reject bad solutions.

The factors that need to be considered by the algorithm are:

1. Each student should be able to contribute to their project [Obj. 2b] – they should have at least one skill that is demanded by the project
2. The demands of the project should be met by the students – the more skills are met by the students, the better the grouping
3. There should be a mix of genders [Obj. 2e] – there must be at least two different genders

These factors are of different levels of importance, so their contribution to the fitness value needs to be weighted. After talking with my third party, we decided on weights of 0.6, 0.25, and 0.15 respectively.

The fitness value for each group is then averaged to form a fitness for the whole set of groupings.

2.4.3 Diagrams

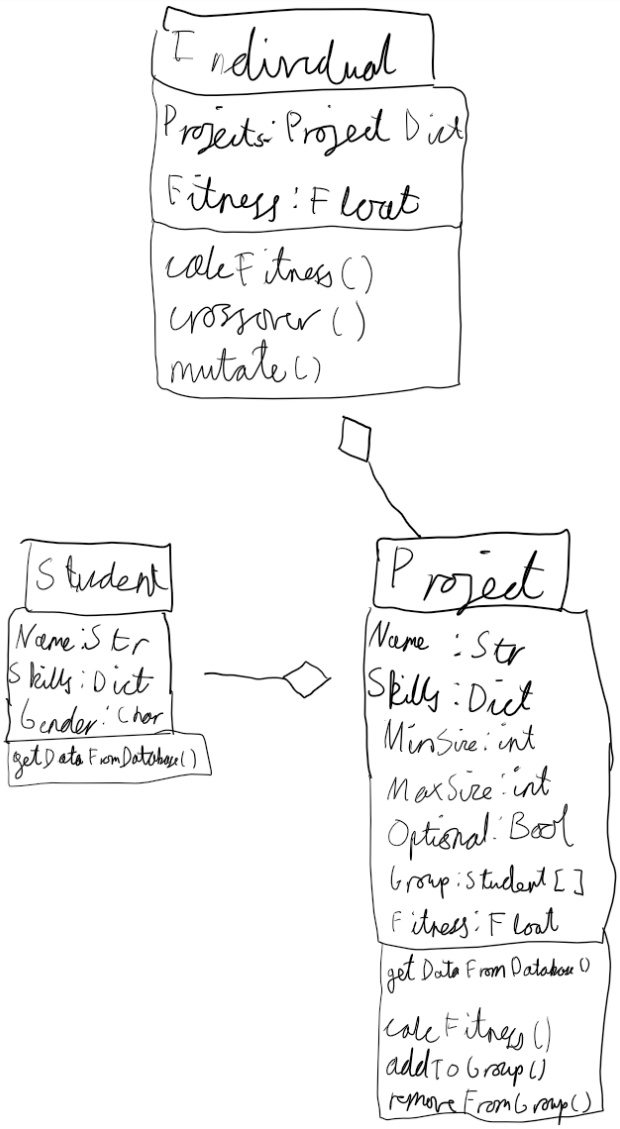


Figure 18: A class diagram showing the attributes and methods of the classes used by the genetic algorithm, and the relationships between each class.

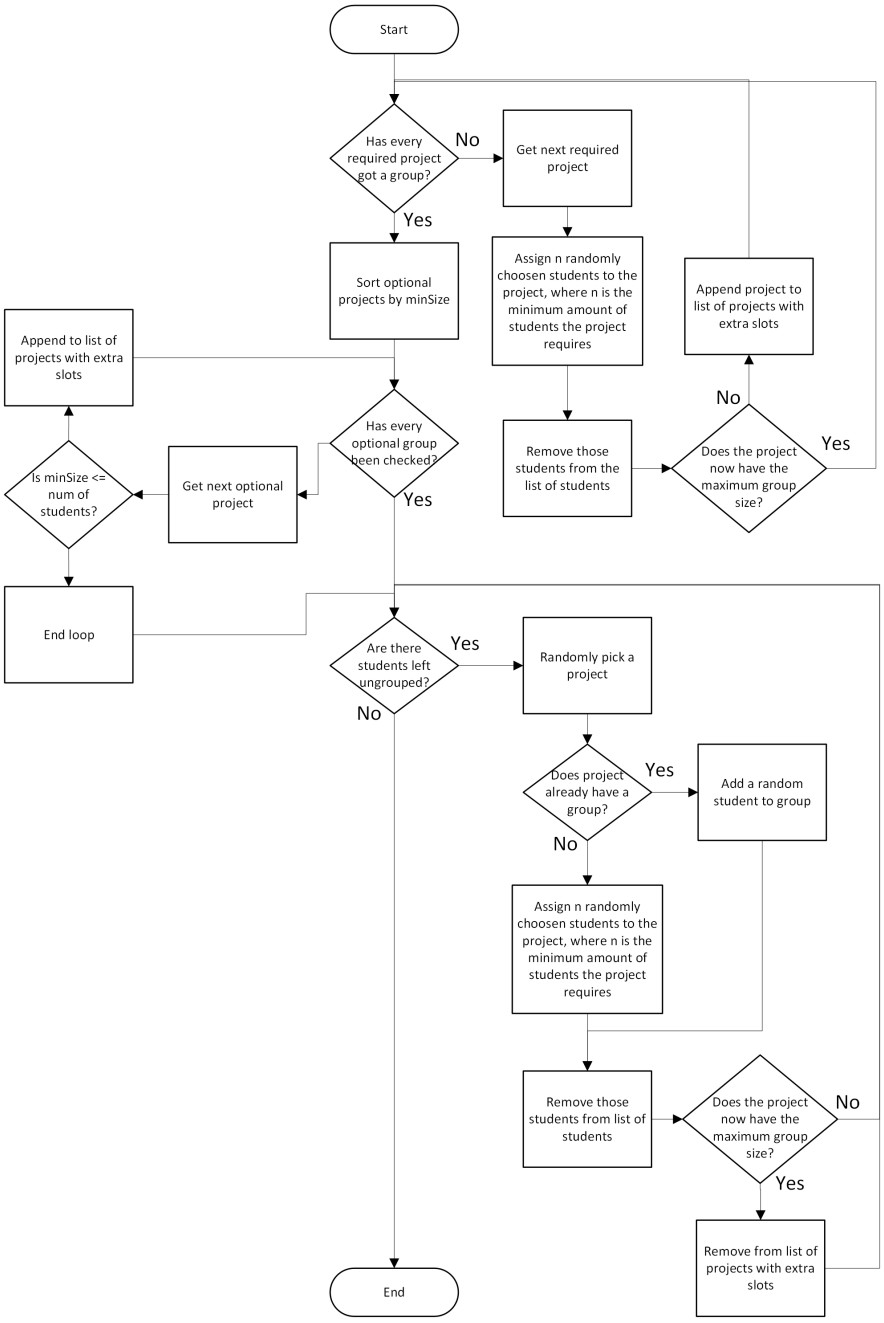


Figure 19: A flowchart for the GenInitialGroups subroutine on line 33 of the above pseudocode. It takes a list of all projects and a list of all students as parameters and returns an Individual object.

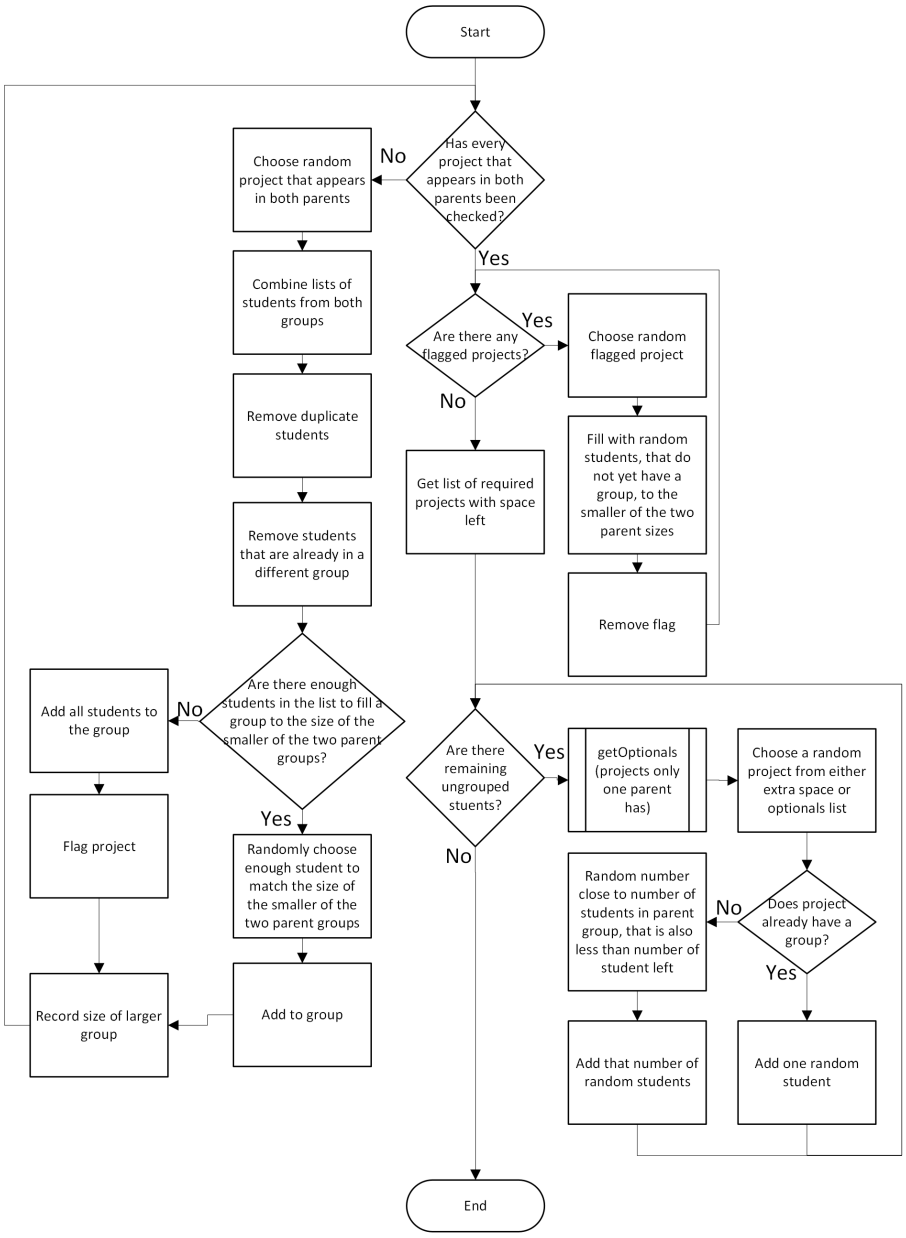


Figure 20: A flowchart for the Crossover method on line 52 of the above pseudocode and in the Individual class in the above class diagram. It takes two Individual objects, and the list of students, as parameters and returns an Individual object.

3 Technical solution

Contents

3.1 main.py 33
3.2 geneticAlgo.py 59
3.3 CreateTables.sql 69

main.py is the main file that is run by the user. It contains the GUI, and calls to the functions and classes in the other files when needed. All the pages outlined above [Fig. 10] have their own classes, whose objects are created dynamically and then stored in the window class. The page classes' attributes are mostly widgets, both tkinter widgets and custom widgets classes (such as Header, ScrollableFrame, SelectionBox, and ProjectBox), or the attributes store the result of user interaction, such as the state of the checkboxes on the project page. The methods all code for the processing and outputting the result of human interaction with the GUI.

geneticAlgo.py contains the code for genetic algorithm that calculates the groups. The subroutines follow the design of the above pseudocode [Listing 1] and the classes follow the design of the above class diagram [Fig. 18].

Criteria

Criteria	Location
Complex data model in database (eg several interlinked tables)	CreateTables.sql file
Complex user-defined use of OOP model	-Inheritance. E.g. projectBox inherits from selectionBox [Line 910] -Composition e.g. ScrollableFrame in StudentPage [Line 406] -Polymorphism e.g. the tkraise method in HomePage [Line 89] -Abstract methods e.g. get_data in selectionBox [Line 903]
Cross-table parameterised SQL	Line 148
Dynamic generation of objects based on complex user-defined use of OOP model	The show_frame method of the Window class [Line 51]
Text files	write_csv method of the SettingsPage class [Line 822]
Writing and reading from files	-Writing e.g. write_csv method of the SettingsPage class [Line 822] -Reading e.g. import_projects method of the ProjectPage class [Line 280]

Dictionaries	frames attribute in Window class [Line 38]

3.1 main.py

```
1  import tkinter as tk
2  from tkinter import ttk, filedialog, messagebox
3  import sqlite3 as sql
4  import csv
5  import re
6  import geneticAlgo as ga
7  #from algoClasses import Student, Project, Individual
8
9  #=====GLOBAL CONSTANTS=====
10
11 BG_COLOUR="#FFFFFF"
12 HEADER_COLOUR="#FFDB32"
13 PASTEL_COLOURS=("#FFBABB","#FFDFBA","#FFFFBA","#BAFFC9","#BAE1FF","#
       ↪ FFBAFF")
14 LARGE_FONT=("Arial",30)
15 DATABASE = "Code/database.db"
16 #SQLFILEHEADER="SQLite format 3"
17
18 #=====ROOT WINDOW=====
19 #Constants for styling included within
20
21 class Window(tk.Tk):
22     def __init__(self):
23         #Set up window
24         tk.Tk.__init__(self)
25
26         self.state('zoomed')
27         self.geometry(f"{self.winfo_screenwidth()-10}x{self.
       ↪ winfo_screenheight()}")
28         self.title('Student Grouping Tool')
29         self.config(background=BG_COLOUR)
30
31         #=====STYLES=====
32
33         HEADER_BUTTON_STYLE=ttk.Style()
34         HEADER_BUTTON_STYLE.configure("Header.TButton",background=
       ↪ HEADER_COLOUR)
35
36         #================
37         #Dictionary that stores all loaded pages
38         self.frames={}
39         #Load the homepage first
```

```
40          self.show_frame(HomePage)
41
42      #Method for changing currently open page
43      def show_frame(self, f,*args,**kwargs):
44          #If the page is already loaded, show it
45          if f in self.frames.keys():
46              frame = self.frames[f]
47          #Otherwise, load it then show it
48          else:
49              frame = f(self)
50              self.frames[f] = frame
51              frame.place(relx=0,rely=0,relwidth=1,relheight=1)
52          frame.tkraise(*args,**kwargs)
53
54  #=====PAGES=====
55
56  class HomePage(tk.Frame):
57      def __init__(self,parent):
58          #Set up home page layout
59          tk.Frame.__init__(self, parent)
60
61          self.settingsButton = ttk.Button(self,text="Settings",command=
    ↪ lambda:parent.show_frame(SettingsPage))
62          self.settingsButton.pack()
63
64          self.label = ttk.Label(self, text="Welcome to the Student
    ↪ Grouping Tool!", font=LARGE_FONT)
65          self.label.pack()
66
67          self.projectButton = ttk.Button(self,text="Choose Projects",
    ↪ command=lambda:parent.show_frame(ProjectPage))
68          self.projectButton.pack()
69
70          self.studentButton = ttk.Button(self,text="Choose Students",
    ↪ command=lambda:parent.show_frame(StudentPage))
71          self.studentButton.pack()
72
73          self.runButton = ttk.Button(self,text="Create Groups",command=
    ↪ self.get_data_for_algo,state="disabled")
74          self.runButton.pack()
75
76          self.warning = ttk.Label(self,text="Must choose projects and
    ↪ students first")
77
78
79      #Method override that runs once everytime the page is opened
80      def tkraise(self,*args,**kwargs):
81          tk.Frame.tkraise(self)
82
```

```
83          #If at least one student and one project has been selected, allow
    ↪  the run button to be clickable and remove the warning message
84          if ProjectPage in self.master.frames and StudentPage in self.
    ↪ master.frames and self.master.frames[ProjectPage].
    ↪ check_for_selection() and self.master.frames[StudentPage].
    ↪ check_for_selection():
85              self.runButton["state"]="enabled"
86              self.warning.pack_forget()
87
88          #Otherwise, disable the run button and display the warning
    ↪ message
89          else:
90              self.runButton["state"]="disabled"
91              self.warning.pack()
92
93      def get_data_for_algo(self,*args,**kwargs):
94          database=DATABASE
95          projects=self.master.frames[ProjectPage].get_selected()
96          students=self.master.frames[StudentPage].get_selected()
97          settings=self.master.frames[SettingsPage].get_settings()
98          fitness=settings["fitness"]
99          iterations=settings["iterations"]
100         popSize=settings["popSize"]
101
102         self.master.show_frame(LoadingPage,database,students,projects,
    ↪ fitness,iterations,popSize)
103
104
105
106
107
108 class ProjectPage(tk.Frame):
109     def __init__(self,parent):
110         #Set up the project page layout
111         tk.Frame.__init__(self,parent)
112
113         self.header=Header(self,"Projects",HEADER_COLOUR)
114         self.header.pack(fill="x")
115
116         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
117         self.backButton.pack(side="right")
118
119         self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_projects)
120         self.importButton.pack()
121
122         self.addButton=ttk.Button(self,text="New project",command=lambda:
    ↪ self.create_entry_window("New"))
123         self.addButton.pack()
```

```
124
125         #self.separator=ttk.Separator(self,orient="horizontal")
126         #self.separator.pack(fill="x",expand=True)
127
128         self.boxesFrame=ScrollableFrame(self)
129         self.boxesFrame.canvas.pack(fill="both",expand=True)
130         self.update()
131
132         #Get and show project data from the database
133         self.projectBoxes={}
134         self.fetch_from_db(DATABASE)
135
136
137     def fetch_from_db(self,path):
138         #Fetch all project data from a database
139         query="SELECT * from Projects LEFT JOIN ProjectSkills using (
    ↪ ProjectID)"
140         with sql.connect(path) as conn:
141             cur = conn.cursor()
142             cur.execute(query)
143             projects=cur.fetchall()
144
145             for project in projects:
146                 ID=project[0]
147                 #When a project has multiple skills, the ID is repeated
    ↪ in the recieved tuple for each skill
148                 if ID in self.projectBoxes:
149                     #The next skill is appended to the existing list of
    ↪ skills for this ID
150                     data=self.projectBoxes[ID].get_data()
151
152                     skills=data["skills"]
153                     skills.append(project[5])
154                     self.projectBoxes[ID].edit_data(skills=skills)
155
156                 #If the project has not been added to the dictionary yet,
    ↪  add it
157                 else:
158                     #Unpack the data, keeping data that should not be of
    ↪ type string spearate
159                     *data,optional,skills=project
160                     #Convert to correct data type
161                     optional=bool(optional)
162                     skills=[skills]
163                     #Store data and display data on UI
164                     self.projectBoxes[ID]=projectBox(self.boxesFrame,self
    ↪ ,*data,optional,skills)
165                     self.projectBoxes[ID].pack()
166
167
```

```
168     #Method override as a work-around for a scrolling bug
169     def update(self):
170         self.master.state('normal')
171         tk.Frame.update(self)
172         self.master.state('zoomed')
173 
174     #Iterates through each project, returning true as soon as it finds a
        ↪ selected project. Returns false when no project is selected.
175     def check_for_selection(self):
176         for box in self.projectBoxes:
177             if self.projectBoxes[box].check_selection():
178                 return True
179         return False
180 
181     def get_selected(self):
182         sel=set()
183         for box in self.projectBoxes:
184             if self.projectBoxes[box].check_selection():
185                 sel.add(box)
186         return sel
187 
188 
189     def create_entry_window(self,mode,**kwargs):
190         name=tk.StringVar()
191         maxSize=tk.StringVar()
192         minSize=tk.StringVar()
193         optional=tk.BooleanVar()
194         skills=tk.StringVar()
195         data={"name":name,"maxSize":maxSize,"minSize":minSize,"optional":
        ↪ optional,"skills":skills}
196 
197         if mode == "Edit":
198             popupWin=entryPopupWin(self,"Edit Project")
199             name.set(kwargs["name"])
200             maxSize.set(kwargs["maxSize"])
201             minSize.set(kwargs["minSize"])
202             optional.set(kwargs["optional"])
203             skills.set(", ".join(kwargs["skills"]))
204             data["ID"]=kwargs["ID"]
205         else:
206             popupWin=entryPopupWin(self,"New project")
207 
208         popupWin.grab_set()
209         popupWin.add_string_entry("Name:",name)
210 
211         popupWin.add_int_entry("Maximum size allowed for this project's
        ↪ group:",maxSize)
212 
213         popupWin.add_int_entry("Minimum size allowed for this project's
        ↪ group:",minSize)
```

```
214
215         popupWin.add_checkbox("Optional:",optional)
216
217         popupWin.add_string_entry("Skills, each one separated by a comma:
    ↪ ",skills)
218
219         popupWin.add_save_cancel(popupWin,mode,**data)
220
221     def validate_entry(self,name,maxSize,minSize,optional,skills):
222         if name != "" and maxSize != "" and minSize != "" and optional !=
    ↪  "" and skills != "":
223             try:
224                 maxSize = int(maxSize)
225                 minSize = int(minSize)
226             except:
227                 return {"valid":False,"reason":"Minimum group size and
    ↪ maximum group size must be integers"}
228             else:
229                 if minSize>0:
230                     if maxSize>=minSize:
231                         return {"valid":True}
232                     else:
233                         return {"valid":False,"reason":"Maximum group
    ↪ size must be greater than or equal to minimum group size."}
234                 else:
235                     return {"valid":False,"reason":"Minimum group size
    ↪ must be greater than 0."}
236         else:
237             return {"valid":False,"reason":"All entries must be filled."}
238
239     def save_entry(self,window,mode,**kwargs):
240         name=kwargs["name"].get()
241         maxSize=kwargs["maxSize"].get()
242         minSize=kwargs["minSize"].get()
243         optional=kwargs["optional"].get()
244         skills=kwargs["skills"].get()
245
246         valid=self.validate_entry(name,maxSize,minSize,optional,skills)
247         if valid["valid"]:
248             #Separate skills into a list, remove whitespace, and add to a
    ↪  set to remove duplicates
249             skillList=skills.split(",")
250             skills=set()
251             for i in range(len(skillList)):
252                 skills.add(skillList[i].strip())
253
254             if mode == "Edit":
255                 self.edit_project(kwargs["ID"],name,maxSize,minSize,
    ↪ optional,skills)
256             else:
```

```
257                 self.new_project(name,maxSize,minSize,optional,skills)
258
259             #Destroy the popup window
260             window.destroy()
261
262         else:
263             messagebox.showerror(title="Error",message=valid["reason"])
264
265
266     def import_projects(self):
267
268
269         def importtxt(path):
270             file=open(path,mode="r")
271             file.close()
272
273
274         def importcsv(path):
275             file=open(path,newline='')
276             reader=csv.reader(file)
277             isRow1=True
278             for row in reader:
279                 if isRow1 and not row==["name, maxSize, minSize, optional
    ↪ , skills"]:
280                     messagebox.showerror(title="Import Error",message="
    ↪ CSV File not in correct format.")
281                     break
282                 isRow1=False
283
284                 valid=self.validate_entry(*row)
285                 if valid["valid"]:
286                     #Separate skills into a list, remove whitespace, and
    ↪ add to a set to remove duplicates
287                     skillList=row[4].split(",")
288                     skills=set()
289                     for i in range(len(skillList)):
290                         skills.add(skillList[i].strip())
291
292                     self.new_project(row[0],row[1],row[2],row[3],skills)
293                 else:
294                     messagebox.showerror(title="Import Error",message=f"
    ↪ Import failed. {valid["reason"]}")
295
296
297         def importdb(path):
298             return
299
300
301         #Show an "Open" dialog box and return file directory
302         path = filedialog.askopenfilename()
```

```
303         #Validation to ensure the user selected a valid file type
304         if path != None:
305             match path[-4:]:
306                 #case ".txt":
307                     #importtxt(path)
308                 case ".csv":
309                     importcsv(path)
310                 case _:
311                     #if checkForDB(path):
312                         #importdb(path)
313                     #else:
314                     messagebox.showerror(title="Import Error", message="
    ↪ File type unrecognised!")
315 
316         else:
317             messagebox.showerror(title="Import Error", message="No file
    ↪ selected!")
318 
319 
320 
321     #Add the inputted data to the Project and Project Skills tables in
    ↪ the database
322     def new_project(self,name,maxSize,minSize,optional,skills):
323         mainquery="INSERT INTO Projects (Name, MaxSize, MinSize, Optional
    ↪ ) VALUES (?,?,?,?)"
324         skillsquery="INSERT INTO ProjectSkills (ProjectID, Skill) VALUES
    ↪ (?,?)"
325         with sql.connect(DATABASE) as conn:
326             cur = conn.cursor()
327 
328             cur.execute(mainquery,(name,maxSize,minSize,optional))
329 
330             #Fetch the autogenerated ID for this student
331             ID=cur.lastrowid
332 
333             for skill in skills:
334                 cur.execute(skillsquery,(ID, skill))
335 
336             conn.commit()
337 
338         #Display new project on the GUI
339         self.projectBoxes[ID]=projectBox(self.boxesFrame,self,ID,name,
    ↪ maxSize,minSize,optional,skills)
340         self.projectBoxes[ID].pack()
341 
342         #Work-around for a scrollbar bug
343         self.update()
344 
345     def edit_project(self,ID,name,maxSize,minSize,optional,skills):
346         edited=False
```

```
347         box=self.projectBoxes[ID]
348         oldData=box.get_data()
349 
350         #Only updates if the new data is different from the old data
351         with sql.connect(DATABASE) as conn:
352             cur = conn.cursor()
353 
354             if name != oldData["name"]:
355                 edited=True
356 
357                 query = "UPDATE Projects SET Name = ? WHERE ProjectID = ?
    ↪ "
358                 cur.execute(query,(name,ID))
359 
360                 box.edit_data(name=name)
361 
362             if maxSize != oldData["maxSize"]:
363                 edited=True
364 
365                 query="UPDATE Projects SET MaxSize = ? WHERE ProjectID =
    ↪ ?"
366                 cur.execute(query,(maxSize,ID))
367 
368                 box.edit_data(maxSize=maxSize)
369 
370             if minSize != oldData["minSize"]:
371                 edited=True
372 
373                 query="UPDATE Projects SET MinSize = ? WHERE ProjectID =
    ↪ ?"
374                 cur.execute(query,(minSize,ID))
375 
376                 box.edit_data(minSize=minSize)
377 
378             if optional != oldData["optional"]:
379                 edited=True
380 
381                 query="UPDATE Projects SET Optional = ? WHERE ProjectID =
    ↪  ?"
382                 cur.execute(query,(optional,ID))
383 
384                 box.edit_data(optional=optional)
385 
386             if skills != oldData["skills"]:
387                 edited=True
388 
389                 cur.execute("DELETE FROM ProjectSkills WHERE ProjectID =
    ↪ ?",(ID,))
390                 query = "INSERT INTO ProjectSkills (ProjectID, Skill)
    ↪ VALUES (?,?)"
```

```
391                 for skill in skills:
392                     cur.execute(query,(ID,skill))
393 
394                 box.edit_data(skills=skills)
395 
396             if edited:
397                 self.update()
398                 conn.commit()
399 
400     def editBox(self,boxID):
401         box=self.projectBoxes[boxID]
402         data=box.get_data()
403         self.create_entry_window("Edit",ID=data["ID"],name=data["name"],
    ↪ maxSize=data["maxSize"],minSize=data["minSize"], optional=data["
    ↪ optional"], skills=data["skills"])
404 
405     def deleteBox(self,boxID):
406         #Delete from database
407         query1 = "DELETE FROM Projects WHERE ProjectID=?"
408         query2 = "DELETE FROM ProjectSkills WHERE ProjectID=?"
409         with sql.connect(DATABASE) as conn:
410             cur = conn.cursor()
411             cur.execute(query1,(boxID,))
412             cur.execute(query2,(boxID,))
413             conn.commit()
414         #Delete from UI
415         self.projectBoxes[boxID].pack_forget()
416         #Delete from internal store
417         del self.projectBoxes[boxID]
418 
419         self.update()
420 
421 
422 
423 class StudentPage(tk.Frame):
424     def __init__(self,parent):
425         #Set up student page layout
426         tk.Frame.__init__(self, parent)
427 
428         self.header=Header(self,"Students",HEADER_COLOUR)
429         self.header.pack(fill="x")
430 
431         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
432         self.backButton.pack(side="right")
433 
434         self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_students)
435         self.importButton.pack()
436 
```

```
437         self.addButton=ttk.Button(self,text="New student",command=lambda:
    ↪ self.create_entry_window("New"))
438         self.addButton.pack()
439 
440         self.boxesFrame=ScrollableFrame(self)
441         self.boxesFrame.canvas.pack(fill="both",expand=True)
442         self.update()
443 
444         self.studentBoxes={}
445 
446         #Fetch all student data from the database
447         studentquery="SELECT * from Students LEFT JOIN StudentSkills
    ↪ using (StudentID)"
448         with sql.connect(DATABASE) as conn:
449             cur = conn.cursor()
450             cur.execute(studentquery)
451             students=cur.fetchall()
452 
453             for student in students:
454                 ID=student[0]
455                 #When a student has multiple skills, the ID is repeated
    ↪ in the recieved tuple for each skill
456                 if ID in self.studentBoxes:
457                     #The next skill is appended to the existing list of
    ↪ skills for this ID
458                     data=self.studentBoxes[ID].get_data()
459                     skills=data["skills"]
460                     skills.append(student[3])
461                     self.studentBoxes[ID].edit_data(skills=skills)
462                 #If the student has not been added to the dictionary yet,
    ↪  add it
463                 else:
464                     #Unpack the data, keeping data that should not be of
    ↪ type string spearate
465                     *data,skills=student
466                     #Convert to correct data type
467                     skills=[skills]
468                     #Store data and display on UI
469                     self.studentBoxes[ID]=studentBox(self.boxesFrame,self
    ↪ ,*data,skills)
470                     self.studentBoxes[ID].pack()
471 
472     #Method override as a work-around for a scrolling bug
473     def update(self):
474         self.master.state('normal')
475         tk.Frame.update(self)
476         self.master.state('zoomed')
477 
478     def check_for_selection(self):
479         for box in self.studentBoxes:
```

```
480             if self.studentBoxes[box].check_selection():
481                 return True
482         return False
483 
484     def get_selected(self):
485         sel=set()
486         for box in self.studentBoxes:
487             if self.studentBoxes[box].check_selection():
488                 sel.add(box)
489         return sel
490 
491     def validate_entry(self,name,gender,skills):
492         if name != "" and gender != "" and skills != "":
493             return {"valid":True}
494         else:
495             return {"valid":False,"reason":"All entries must be filled."}
496 
497 
498     def import_students(self):
499 
500         def importcsv(path):
501             file=open(path,newline='')
502             reader=csv.reader(file)
503             isRow1=True
504             for row in reader:
505                 if isRow1 and not row==["name, gender, skills"]:
506                     messagebox.showerror(title="Import Error",message="
    ↪ CSV File not in correct format.")
507                     break
508                 isRow1=False
509 
510                 valid=self.validate_entry(*row)
511                 if valid["valid"]:
512                     #Separate skills into a list, remove whitespace, and
    ↪ add to a set to remove duplicates
513                     skillList=row[2].split(",")
514                     skills=set()
515                     for i in range(len(skillList)):
516                         skills.add(skillList[i].strip())
517 
518                     self.new_project(row[0],row[1],skills)
519                 else:
520                     messagebox.showerror(title="Import Error",message=f"
    ↪ Import failed. {valid["reason"]}")
521 
522 
523         #Show an "Open" dialog box and return an open file object in read
    ↪  mode
524         path = filedialog.askopenfilename()
525         #Validation to ensure the user selected a valid file type
```

```
526         if path != None:
527             match path[-4:]:
528                 #case ".txt":
529                     #importtxt(path)
530                 case ".csv":
531                     importcsv(path)
532                 case _:
533                     #if checkForDB(path):
534                         #importdb(path)
535                     #else:
536                     messagebox.showerror(title="Import Error", message="
    ↪ File type unrecognised!")
537 
538         else:
539             messagebox.showerror(title="Import Error", message="No file
    ↪ selected!")
540 
541     def create_entry_window(self,mode,**kwargs):
542         name=tk.StringVar()
543         gender=tk.StringVar()
544         skills=tk.StringVar()
545         data={"name":name,"gender":gender,"skills":skills}
546 
547         if mode == "Edit":
548             popupWin=entryPopupWin(self, "Edit student")
549             name.set(kwargs["name"])
550             gender.set(kwargs["gender"])
551             skills.set(", ".join(kwargs["skills"]))
552             data["ID"]=kwargs["ID"]
553         else:
554             popupWin=entryPopupWin(self,"New student")
555 
556         popupWin.grab_set()
557 
558         popupWin.add_string_entry("Name:", name)
559 
560         popupWin.add_string_entry("Gender:",gender)
561 
562         popupWin.add_string_entry("Skills, each one separated by a comma:
    ↪ ",skills)
563 
564         popupWin.add_save_cancel(popupWin,mode,**data)
565 
566     def save_entry(self,window,mode,**kwargs):
567         name=kwargs["name"].get()
568         gender=kwargs["gender"].get()
569         skills=kwargs["skills"].get()
570 
571         #Validation to ensure all entries meet standards
572         valid=self.validate_entry(name,gender,skills)
```

```
573             if valid["valid"]:
574                 #Separate skills into a list, remove whitespace, add to set
    ↪ to remove duplicates
575                 skillList=skills.split(",")
576                 skills=set()
577                 for i in range(len(skillList)):
578                     skills.add(skillList[i].strip())
579 
580                 if mode == "Edit":
581                     self.edit_student(kwargs["ID"],name,gender,skills)
582                 else:
583                     self.new_student(name,gender,skills)
584 
585                 #Destroy the popup window
586                 window.destroy()
587             else:
588                 messagebox.showerror(title="Error",message=valid["reason"])
589 
590 
591     #Add the inputted data to the Student and StudentSkills tables in the
    ↪  database
592     def new_student(self,name,gender,skills):
593         mainquery="INSERT INTO Students (Name, Gender) VALUES (?,?)"
594         skillsquery="INSERT INTO StudentSkills (StudentID, Skill) VALUES
    ↪ (?,?)"
595         with sql.connect(DATABASE) as conn:
596             cur = conn.cursor()
597 
598             cur.execute(mainquery,(name,gender))
599 
600             #Fetch the autogenerated ID for this student
601             ID=cur.lastrowid
602 
603             for skill in skills:
604                 cur.execute(skillsquery,(ID, skill))
605 
606             conn.commit()
607 
608         #Display new student on the GUI
609         self.studentBoxes[ID]=studentBox(self.boxesFrame,self,ID,name,
    ↪ gender,skills)
610         self.studentBoxes[ID].pack()
611 
612         #Scroll bar bug work-around
613         self.update()
614 
615 
616     def edit_student(self,ID,name,gender,skills):
617         edited=False
618 
```

```
619         box=self.studentBoxes[ID]
620         oldData=box.get_data()
621 
622         #Only updates if the new data is different from the old data
623         with sql.connect(DATABASE) as conn:
624             cur = conn.cursor()
625 
626             if name != oldData["name"]:
627                 edited=True
628 
629                 query = "UPDATE Students SET Name = ? WHERE StudentID = ?
    ↪ "
630                 cur.execute(query,(name,ID))
631 
632                 box.edit_data(name=name)
633 
634             if gender != oldData["gender"]:
635                 edited=True
636 
637                 query="UPDATE Students SET Gender = ? WHERE StudentID = ?
    ↪ "
638                 cur.execute(query,(gender,ID))
639 
640                 box.edit_data(gender=gender)
641 
642             if skills != oldData["skills"]:
643                 edited=True
644 
645                 cur.execute("DELETE FROM StudentSkills WHERE StudentID =
    ↪ ?",(ID,))
646                 query = "INSERT INTO StudentSkills (StudentID, Skill)
    ↪ VALUES (?,?)"
647                 for skill in skills:
648                     cur.execute(query,(ID,skill))
649 
650                 box.edit_data(skills=skills)
651 
652             if edited:
653                 self.update()
654                 conn.commit()
655 
656     def editBox(self,boxID):
657         box=self.studentBoxes[boxID]
658         data=box.get_data()
659         self.create_entry_window("Edit", ID=data["ID"],name=data["name"],
    ↪ gender=data["gender"],skills=data["skills"])
660 
661     def deleteBox(self,boxID):
662         #Delete from database
663         query1 = "DELETE FROM Students WHERE StudentID=?"
```

```
664         query2 = "DELETE FROM StudentSkills WHERE StudentID=?"
665         with sql.connect(DATABASE) as conn:
666             cur = conn.cursor()
667             cur.execute(query1,(boxID,))
668             cur.execute(query2,(boxID,))
669             conn.commit()
670         #Delete from UI
671         self.studentBoxes[boxID].pack_forget()
672         #Delete from internal store
673         del self.studentBoxes[boxID]
674 
675         self.update()
676 
677 
678 
679 class LoadingPage(tk.Frame):
680     def __init__(self,parent):
681         #Set up loading page layout
682         tk.Frame.__init__(self, parent)
683 
684         self.label = ttk.Label(self, text="Grouping in progress...")
685         self.label.pack()
686 
687         self.cancelButton=ttk.Button(self,text="Cancel",command=lambda:
    ↪ parent.show_frame(HomePage))
688         self.cancelButton.pack()
689 
690     #Method override that runs once everytime the page is opened
691     def tkraise(self, database, studentIDs, projectIDs, tgtFit, maxIters,
    ↪  popSize,*args,**kwargs):
692         tk.Frame.tkraise(self)
693         results=ga.run(database,studentIDs,projectIDs,tgtFit,maxIters,
    ↪ popSize)
694         if results["status"]==-1:
695             messagebox.showerror("Error",results["error"])
696         elif results["status"]==0:
697             results=results["results"]
698             self.master.show_frame(ResultsPage,results)
699         else:
700             messagebox.showerror("Error","An unknown eror occured")
701 
702 
703 class ResultsPage(tk.Frame):
704     def __init__(self,parent):
705         #Set up results page layout
706         tk.Frame.__init__(self, parent)
707 
708         self.results=None
709         self.resultBoxes={}
710 
```

```
711         self.header=Header(self,"Results",HEADER_COLOUR)
712         self.header.pack(fill="x")
713 
714         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
715         self.homeButton.pack(side="right")
716 
717         self.registerButton=ttk.Button(self.header,text="Switch to
    ↪ register layout",style="Header.TButton",command=lambda:parent.
    ↪ show_frame(RegisterPage,self.results))
718         self.registerButton.pack(side="right")
719 
720         #Allows scrolling through the boxes
721         self.boxesFrame=ScrollableFrame(self)
722         self.boxesFrame.canvas.pack(fill="both",expand=True)
723         self.update()
724 
725     def tkraise(self,results,*args,**kwargs):
726         tk.Frame.tkraise(self)
727 
728         while len(self.resultBoxes)>0:
729             ID=next(iter(self.resultBoxes))
730             self.resultBoxes[ID].pack_forget()
731             del self.resultBoxes[ID]
732 
733         self.results=results
734         #Stores the boxes shown in the scrollable frame
735         self.resultBoxes={}
736 
737         #Create the boxes and display project data in them
738         for projectID in results:
739             result=results[projectID]
740             self.resultBoxes[projectID]=resultBox(self.boxesFrame,self,
    ↪ projectID,result["name"],result["students"],result["size"],result
    ↪ ["genders"],result["skills"])
741             self.resultBoxes[projectID].pack()
742 
743         self.update()
744 
745     #Method override as a work-around for a scrolling bug
746     def update(self):
747         self.master.state('normal')
748         tk.Frame.update(self)
749         self.master.state('zoomed')
750 
751 
752 class RegisterPage(tk.Frame):
753     def __init__(self,parent):
754         #Set up register page layout
755         tk.Frame.__init__(self, parent)
```

```
756         self.columnconfigure(0,weight=1)
757         self.columnconfigure(1,weight=1)
758         self.columnconfigure(2,weight=2)
759 
760         self.results=None
761         self.registerBoxes={}
762 
763         self.header=Header(self,"Register",HEADER_COLOUR)
764         self.header.grid(column=0,row=0,columnspan=3,sticky="EW")
765 
766         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
767         self.homeButton.pack(side="right")
768 
769         self.resultsButton=ttk.Button(self.header,text="Switch to results
    ↪  layout",style="Header.TButton",command=lambda:parent.show_frame(
    ↪ ResultsPage,self.results))
770         self.resultsButton.pack(side="right")
771 
772         #Allows scrolling through the boxes
773         self.boxesFrame=ScrollableFrame(self)
774         self.boxesFrame.canvas.grid(column=0,row=1,sticky="NES",rowspan=6
    ↪ ,columnspan=3)
775         self.update()
776 
777     #Method override as a work-around for a scrolling bug
778     def update(self):
779         self.master.state('normal')
780         tk.Frame.update(self)
781         self.master.state('zoomed')
782 
783     def tkraise(self,results,*args,**kwargs):
784         tk.Frame.tkraise(self)
785 
786         while len(self.registerBoxes)>0:
787             ID=next(iter(self.registerBoxes))
788             self.registerBoxes[ID].grid_forget()
789             del self.registerBoxes[ID]
790 
791         self.results=results
792         #Stores the boxes shown in the scrollable frame
793         self.registerBoxes={}
794 
795         #Create the boxes and display project data in them
796         for projectID in results:
797             result=results[projectID]
798             self.registerBoxes[projectID]=registerBox(self.boxesFrame,
    ↪ self,projectID,result["name"],result["students"])
799             self.registerBoxes[projectID].grid()
800 
```

```
801         self.update()
802 
803 
804 class SettingsPage(tk.Frame):
805     def __init__(self,parent,fitness,iterations,popSize):
806         #Set up swttings page layout
807         tk.Frame.__init__(self,parent)
808         self.fitness=fitness
809         self.iterations=iterations
810         self.popSize=popSize
811 
812         self.header=Header(self,"Settings",HEADER_COLOUR)
813         self.header.pack(fill="x")
814 
815         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=self.go_back)
816         self.backButton.pack(side="right")
817 
818         self.fitnessVar=tk.DoubleVar(value=self.fitness)
819         self.iterVar=tk.IntVar(value=self.iterations)
820         self.popSizeVar=tk.IntVar(value=self.popSize)
821 
822         ttk.Button(self,text="Export database", command=self.export_db).
    ↪ pack()
823 
824         #Only allows user to type a decimal number between 0 and 1, no
    ↪ other characters
825         def fitnessCallback(P):
826             if re.match("^0(\\.\\d*)?$",P) or re.match("^1(\\.0*)?$",P)
    ↪ or P == "":
827                 return True
828             return False
829 
830         #Only allows the user to enter positive integers
831         def posIntCallback(P):
832             return str.isdigit(P) or P==""
833 
834         fitnessValidate = (self.register(fitnessCallback), '%P')
835         posIntValidate = (self.register(posIntCallback), '%P')
836 
837         ttk.Label(self, text="Target fitness for genetic algorithm (Must
    ↪ be between 0-1).").pack()
838         ttk.Label(self,text="Higher target fitness means better groups,
    ↪ with 1 being perfect groupings. May not be reached if maximum
    ↪ iterations is reached").pack()
839         ttk.Spinbox(self, textvariable=self.fitnessVar,validate="all",
    ↪ validatecommand=fitnessValidate,from_=0,increment=0.05,to=1).pack
    ↪ ()
840 
```

```
841         ttk.Label(self,text="Maximum iterations the genetic algorithm
    ↪ could complete (may be less if target fitness is reached).").pack
    ↪ ()
842         ttk.Spinbox(self,textvariable=self.iterVar,validate="all",
    ↪ validatecommand=posIntValidate,from_=1,increment=5,to=100).pack()
843 
844         ttk.Label(self,text="Size of populations for genetic algorithm.
    ↪ Greater sizes leads to longer runtime, but better results.").pack
    ↪ ()
845         ttk.Spinbox(self,textvariable=self.popSizeVar,validate="all",
    ↪ validatecommand=posIntValidate,from_=1,increment=10,to=200).pack
    ↪ ()
846 
847         ttk.Button(self, text="Save", command=self.save_settings).pack()
848 
849     #Method override that runs once everytime this page is opened
850     def tkraise(self,*args,**kwargs):
851         self.fitnessVar.set(self.fitness)
852         self.iterVar.set(self.iterations)
853         self.popSizeVar.set(self.popSize)
854         tk.Frame.tkraise(self)
855 
856     def get_settings(self):
857         return {"fitness":self.fitness,"iterations":self.iterations,"
    ↪ popSize":self.popSize}
858 
859     def go_back(self): #Only show message when setting has been changed
860         choice=messagebox.askyesno("Warning","Going back will not save
    ↪ your changes. Do you still wish to proceed?",icon="warning")
861         if choice:
862             self.master.show_frame(HomePage)
863 
864     def validate(self):
865         if int(self.popSizeVar.get())>=4:
866             if int(self.iterVar.get())>0:
867                 return True
868             else:
869                 messagebox.showerror(title="Error",message="Max
    ↪ iterations must be greater than 0")
870         else:
871             messagebox.showerror(title="Error",message="Population size
    ↪ must be greater than or equal to 4")
872             return False
873 
874     #Validate and save settings input
875     def save_settings(self):
876         valid=self.validate()
877         if valid:
878             self.fitness=float(self.fitnessVar.get())
879             self.iterations=int(self.iterVar.get())
```

```
            self.popSize=int(self.popSizeVar.get())
            self.master.show_frame(HomePage)
            #Update settings.json

    def export_db(self): #The ui for selecting details before selecting directory
        self.write_csv()

    #Save database to a csv file
    def write_csv(self):
        filetype=[("CSV file",".csv")]
        file = filedialog.asksaveasfile(filetypes=filetype, defaultextension=filetype)
        if file is not None:
            with sql.connect(DATABASE) as conn:
                cur = conn.cursor()
                #Fetch names of every table
                cur.execute("SELECT name FROM sqlite_schema WHERE type ='table' AND name NOT LIKE 'sqlite_%';")
                tables=cur.fetchall()
                #Fetch all the data from each table
                for table in tables:
                    data=cur.execute("SELECT * FROM "+str(table[0]))

                    #Write to the csv
                    writer = csv.writer(file)
                    writer.writerow("")
                    writer.writerow(table)
                    #Get attribute names
                    cols=[]
                    for col in data.description:
                        cols.append(col[0])
                    writer.writerow(cols)
                    writer.writerows(data)
        else:
            #Warn the user export was not successful
            return
        file.close()

#=====WIDGETS=====

class Header(tk.Frame):
    def __init__(self,parent,text,colour):
        tk.Frame.__init__(self,parent)
        self.config(bg=colour)
        self.label = tk.Label(self, text=text, bg=colour,font=LARGE_FONT)
        self.label.pack(side="left")

class ScrollableFrame(tk.Frame):
    def __init__(self,parent):
```

```
927         #The only tkinter widget which supports both scrollbars and
    ↪ embedded widgets is a canvas
928         self.canvas=tk.Canvas(parent,background="white")
929         tk.Frame.__init__(self,self.canvas)
930 
931         self.scrollbar=tk.Scrollbar(self.canvas,orient="vertical",command
    ↪ =self.canvas.yview)
932 
933         self.canvas.configure(yscrollcommand=self.scrollbar.set)
934 
935         self.scrollbar.pack(side="right",fill="y")
936         self.scrollbar.update()
937 
938         #Assign the scrollbar and scrolling function to the canvas
939         self.canvas.create_window((0,0),window=self,anchor='nw',width=
    ↪ self.winfo_screenwidth()-self.scrollbar.winfo_width())
940         self.canvas.bind("<Configure>",self.scroll)
941 
942 
943     def scroll(self,event):
944         self.canvas.configure(scrollregion=self.canvas.bbox("all"),width=
    ↪ self.winfo_width(),height=self.winfo_height())
945 
946 #Has checkbox, name, edit button, delete button.
947 class selectionBox(tk.Frame):
948     def __init__(self,parent,page,ID,name):
949         tk.Frame.__init__(self,parent,relief="solid",bd=2)
950 
951         self.ID=ID
952         self.name=name
953         self.page=page
954 
955         self.selected=tk.BooleanVar(value=False)
956         self.checkbox=ttk.Checkbutton(self,variable=self.selected,onvalue
    ↪ =True,offvalue=False)
957         self.checkbox.pack()
958 
959         self.nameLabel=ttk.Label(self,text=name)
960         self.nameLabel.pack()
961 
962         self.editButton=ttk.Button(self,text="Edit",command=self.edit)
963         self.editButton.pack()
964 
965         self.deleteButton=ttk.Button(self,text="Delete",command=self.
    ↪ delete)
966         self.deleteButton.pack()
967 
968     #Return whether this item has been selected by the user
969     def check_selection(self):
970         return self.selected.get()
```

```
971
972     #Opens the edit window, which is handled by the page the box is
        ↪ located within
973     def edit(self):
974         self.page.editBox(self.ID)
975
976     #Ask for confirmation before deletion
977     #Deletion is handled by the page the box is located on
978     def delete(self):
979         delChoice=messagebox.askyesno(title="Delete entry", message=f"Are
        ↪  you sure you want to delete {self.name}?")
980         if delChoice:
981             #Ensure deleted data can never get caught up in the algorithm
982             self.selected.set(False)
983             self.page.deleteBox(self.ID)
984
985     #Ensure all boxes follow the same design
986     def pack(self):
987         tk.Frame.pack(self,fill="x",padx=10,pady=5)
988
989     #-----Abstract methods-----
990     def get_data(self):
991         return
992
993     def edit_data(self):
994         return
995
996
997 class projectBox(selectionBox):
998     def __init__(self,parent,page,ID,name,maxSize,minSize,optional,skills
        ↪ ):
999         selectionBox.__init__(self,parent,page,ID,name)
1000        self.maxSize=maxSize
1001        self.minSize=minSize
1002        self.optional=optional
1003        self.skills=skills
1004
1005        self.maxSizeLabel=ttk.Label(self,text=f"Max group size: {maxSize}
        ↪ ")
1006        self.maxSizeLabel.pack()
1007
1008        self.minSizeLabel=ttk.Label(self,text=f"Min group size: {minSize}
        ↪ ")
1009        self.minSizeLabel.pack()
1010
1011        self.optionalLabel=ttk.Label(self,text=f"Optional: {optional}")
1012        self.optionalLabel.pack()
1013
1014        self.skillsLabel=ttk.Label(self,text=f"Skills: {", ".join(skills)
        ↪ }")
```

```
1015            self.skillsLabel.pack()
1016
1017        #Return all the data about the project
1018        def get_data(self):
1019            return {"ID":self.ID,"name":self.name,"maxSize":self.maxSize,"
    ↪ minSize":self.minSize,"optional":self.optional,"skills":self.
    ↪ skills}
1020
1021        #After the user has edited some project data, this method updates its
    ↪  data stores and changes the data shown on the GUI
1022        def edit_data(self,**kwargs):
1023            if "name" in kwargs:
1024                self.name=kwargs["name"]
1025                self.nameLabel.config(text=self.name)
1026            if "maxSize" in kwargs:
1027                self.maxSize=kwargs["maxSize"]
1028                self.maxSizeLabel.config(text=f"Max group size: {self.maxSize
    ↪ }")
1029            if "minSize" in kwargs:
1030                self.minSize=kwargs["minSize"]
1031                self.minSizeLabel.config(text=f"Min group size: {self.minSize
    ↪ }")
1032            if "optional" in kwargs:
1033                self.optional=kwargs["optional"]
1034                self.optionalLabel.config(text=f"Optional: {self.optional}")
1035            if "skills" in kwargs:
1036                self.skills=kwargs["skills"]
1037                self.skillsLabel.config(text=f"Skills: {", ".join(self.skills
    ↪ )}")
1038
1039    class studentBox(selectionBox):
1040        def __init__(self, parent, page, ID, name, gender, skills):
1041            selectionBox.__init__(self,parent,page,ID,name)
1042            self.gender=gender
1043            self.skills=skills
1044
1045            self.genderLabel = ttk.Label(self,text=f"Gender: {gender}")
1046            self.genderLabel.pack()
1047
1048            self.skillsLabel = ttk.Label(self, text= f"Skills: {", ".join(
    ↪ skills)}")
1049            self.skillsLabel.pack()
1050
1051        #Returns all data about the student
1052        def get_data(self):
1053            return {"ID":self.ID,"name":self.name,"gender":self.gender,"
    ↪ skills":self.skills}
1054
1055        #After the user has edited some student data, this method updates its
    ↪  data stores and changes the data shown on the GUI
```

```
1056        def edit_data(self,**kwargs):
1057            if "name" in kwargs:
1058                self.name=kwargs["name"]
1059                self.nameLabel.config(text=self.name)
1060            if "gender" in kwargs:
1061                self.gender=kwargs["gender"]
1062                self.genderLabel.config(text=f"Gender: {self.gender}")
1063            if "skills" in kwargs:
1064                self.skills=kwargs["skills"]
1065                self.skillsLabel.config(text=f"Skills: {", ".join(self.skills
        ↪ )}")
1066
1067    #Customisable pop-up widget where different types of entries and buttons
        ↪ can be added dynamically
1068    class entryPopupWin(tk.Toplevel):
1069        def __init__(self,parent,title):
1070            tk.Toplevel.__init__(self,parent)
1071            self.title(title)
1072
1073        def add_string_entry(self,text,variable):
1074            ttk.Label(self, text=text).pack()
1075            ttk.Entry(self, textvariable=variable).pack()
1076
1077        def add_int_entry(self,text,variable,minimum=0,maximum=100,increment=
        ↪ 1):
1078
1079            #Prevents user from typing non numbers into number-only entry
        ↪ boxes
1080            def intCallback(P):
1081                return str.isdigit(P) or P == ""
1082
1083            intValidate = (self.register(intCallback), '%P')
1084
1085            ttk.Label(self, text=text).pack()
1086            ttk.Spinbox(self, textvariable=variable,validate="all",
        ↪ validatecommand=intValidate,from_=minimum,increment=increment,to=
        ↪ maximum).pack()
1087
1088        def add_checkbox(self,text,variable):
1089            ttk.Label(self, text=text).pack()
1090            ttk.Checkbutton(self, variable=variable, onvalue=True, offvalue=
        ↪ False).pack()
1091
1092        def add_save_cancel(self,window,mode,**kwargs):
1093            ttk.Button(self, text="Save", command=lambda:self.master.
        ↪ save_entry(window,mode,**kwargs)).pack()
1094            ttk.Button(self, text="Cancel", command=self.destroy).pack()
1095
1096
1097    class resultBox(tk.Frame):
```

```
1098        #Used for ensuring adjacent boxes have different colours
1099        count=0
1100        def __init__(self,parent,page,ID,name,students:list,size,genders:dict
        ↪ ,skills:dict):
1101            tk.Frame.__init__(self,parent,relief="solid",bd=2)
1102
1103            #Used for ensuring adjacent boxes have different colours
1104            resultBox.count+=1
1105            self.colour=PASTEL_COLOURS[(resultBox.count-1)%6]
1106            self.config(bg=self.colour)
1107
1108
1109            self.page=page
1110            self.ID=ID
1111
1112            ttk.Label(self,text=name,background=self.colour).pack(anchor="w")
1113            ttk.Separator(self).pack(fill="x")
1114
1115            ttk.Label(self,text=f"Students: {", ".join(students)}",background
        ↪ =self.colour).pack(anchor="w")
1116            ttk.Label(self,text=f"Group size: {size}",background=self.colour)
        ↪ .pack(anchor="w")
1117
1118            ttk.Label(self,text="Genders:",background=self.colour).pack(
        ↪ anchor="w")
1119            for gender in genders:
1120                ttk.Label(self,text=f"  > {gender}: {genders[gender]}",
        ↪ background=self.colour).pack(anchor="w")
1121
1122            ttk.Label(self,text="Skills:",background=self.colour).pack(anchor
        ↪ ="w")
1123            for skill in skills:
1124                ttk.Label(self,text=f"   > {skill}: {skills[skill]}",
        ↪ background=self.colour).pack(anchor="w")
1125
1126        #Ensure this widget always follows the same design
1127        def pack(self):
1128            tk.Frame.pack(self,fill="x",padx=10,pady=5)
1129
1130    class registerBox(tk.Frame):
1131        #Used for ensuring adjacent boxes have different colours
1132        count=0
1133        def __init__(self,parent,page,ID,name,students:list):
1134            tk.Frame.__init__(self,parent,relief="solid",bd=2)
1135
1136            #Used for ensuring adjacent boxes have different colours
1137            registerBox.count+=1
1138            self.colour=PASTEL_COLOURS[(registerBox.count-1)%6]
1139            self.config(bg=self.colour)
1140
```

```
        ttk.Label(self,text=name,background=self.colour).pack(anchor="w")
        ttk.Separator(self).pack(fill="x")

        for student in students:
            ttk.Label(self,text=student,background=self.colour).pack(
    ↪ anchor="w")









#=====SUBROUTINES=====
def checkForDB(path):
    #Database files could have any extension, but they all start with the
    ↪  same first 15 characters.
    file=open(path,"rb")
    line=file.read(15)
    file.close()
    #Convert binary to string
    line="".join(chr(x) for x in line)
    if line == "SQLite format 3":
        return True
    return False


#=====MAIN=====

app=Window()
app.mainloop()
```

3.2 geneticAlgo.py

```
import sqlite3 as sql
import random

#=====CLASSES=====
class Project:
    def __init__(self,ID,name,skills,maxSize,minSize,optional):
        self.ID=ID
        self.name=name
        self.skills=skills
        self.maxSize=maxSize
        self.minSize=minSize
        self.optional=optional
```

```
13
14          self.group=[]
15
16      #Returns count of genders and skills as dicts, and list of students
17      def get_data(self):
18          genders={}
19          skills={}
20          students=[]
21          for student in self.group:
22              if student.gender in genders:
23                  genders[student.gender]+=1
24              else:
25                  genders[student.gender]=1
26
27              for skill in student.skills:
28                  if skill in skills:
29                      skills[skill]+=1
30                  else:
31                      skills[skill]=1
32
33              students.append(student.__str__())
34          return genders,skills,students
35
36      #Returns fitness value as a float between 0-1
37      def getFitness(self):
38          fitness=0.0
39          stuCount = 0 #Records number of students with at least one
    ↪ relevant skill
40          skillsMet=set() #Records the skills the students have that the
    ↪ project demands
41          genderCritMet=False #Records whether or not there are multiple
    ↪ genders in the group
42          for student in self.group:
43              commonSkills=student.skills&self.skills
44              #Each student should have one project that is demanded by the
    ↪  project
45              if len(commonSkills)>=1:
46                  stuCount+=1
47
48              #The demands of the project should be met by the students
49              skillsMet=skillsMet|commonSkills
50
51              #There should be at least two different genders
52              if not genderCritMet and student.gender != self.group[0].
    ↪ gender:
53                  genderCritMet=True
54                  fitness+=0.15
55
56          try:
57              fitness+=(stuCount/len(self.group))*0.6
```

```
        except:
            pass
        finally:
            try:
                fitness+=(len(skillsMet)/len(self.skills))*0.25
            except:
                #If the project has no skill requirements, every group
    ↪ has met every skill requirement so earns the max weight
                fitness+=0.25
        return fitness

    #Adds a student to the project's group
    def addStudent(self,students):
        if type(students)==list:
            self.group+=students

        elif type(students)==Student:
            self.group.append(students)

        else:
            raise TypeError(f"Expected data with type list or Student,
    ↪ got data of type {type(students)}")

    def __str__(self):
        string = self.name +", "
        for student in self.group:
            string += student.__str__() +", "
        return string

    #Allows for the same project to appear in different individuals
    def deepcopy(self):
        return Project(self.ID,self.name,self.skills,self.maxSize,self.
    ↪ minSize,self.optional)


class Student:
    def __init__(self,ID,name,skills,gender):
        self.ID=ID
        self.name=name
        self.skills=skills
        self.gender=gender
    def __str__(self):
        return self.name


class Individual:
    def __init__(self):
        self.projects=dict()
        self.fitness=0.0

```

```
105     #Average the fitness values of each project together to return a
    ↪ float between 0-1
106     def calcFitness(self):
107         if self.fitness==-1:
108             return -1
109         total=0.000
110         for project in self.projects.values():
111             total+=project.getFitness()
112         try:
113             self.fitness = round(total/len(self.projects),3)
114         except:
115             self.fitness = 0.000
116         return self.fitness
117 
118     #Adds a project to the individual's list of projects
119     def addProject(self,projects):
120         if type(projects)==list:
121             for project in projects:
122                 self.projects[project.ID]=project
123 
124         elif type(projects)==Project:
125             self.projects[projects.ID]=projects
126 
127         else:
128             raise TypeError(f"Expected data with type list or Project,
    ↪ got data of type {type(projects)}")
129 
130     def crossover(self,other,students):
131         students=set(students)
132         individual=Individual()
133         flagged=dict()
134         leftover=dict() #Stores the ID of required projects that have not
    ↪  reached the size of the larger of their parents' groups as keys,
    ↪  with that size as values
135         projects1=set(self.projects)
136         projects2=set(other.projects)
137         #Set of projectIDs of projects in both parents
138         required=projects1&projects2
139         #List of projects in only one parent
140         optionals=list(projects1^projects2)
141         optionals=list(map(lambda x:(self.projects|other.projects)[x],
    ↪ optionals))
142 
143         #Assign a group to every project that appears in both parents
144         while len(required)>0:
145             #Projects get groups in a random order
146             projectID=random.choice(list(required))
147             required.remove(projectID)
148             project1=self.projects[projectID]
149             project2=other.projects[projectID]
```

```
150             newProject=Project(projectID,project1.name,project1.skills,
    ↪ project1.maxSize,project1.minSize,project1.optional)
151             #Combine student lists and remove duplicates
152             studentOptions=set(project1.group)|set(project2.group)
153             #Remove students in a different group by only keeping
    ↪ students that remain in the students set
154             studentOptions&=students
155 
156             #Fill the group to the size of the smaller parent group
157             minSize=min(len(project1.group),len(project2.group))
158             maxSize=max(len(project1.group),len(project2.group))
159             if len(studentOptions)>=minSize:
160                 temp=list(studentOptions)
161                 random.shuffle(temp)
162                 group=temp[:minSize]
163                 #Prevent the grouped students from being grouped to
    ↪ another project
164                 students-=set(group)
165                 newProject.addStudent(group)
166                 if maxSize>len(group):
167                     leftover[projectID]=maxSize
168 
169             #In case too many students from the parent groups have
    ↪ already been assigned to different projects
170             else:
171                 students-=studentOptions
172                 newProject.addStudent(list(studentOptions))
173                 flagged[projectID]=minSize-len(studentOptions)
174 
175             individual.addProject(newProject)
176 
177         #Ensure every required project has been filled
178         while len(flagged)>0:
179             projectID=random.choice(list(flagged.keys()))
180             size=flagged[projectID]
181             del flagged[projectID]
182             temp=list(students)
183             random.shuffle(temp)
184             group=temp[:size]
185             students-=set(group)
186             individual.projects[projectID].addStudent(group)
187 
188 
189 
190         while len(students)>0:
191             optionals=get_optionals(optionals,len(students))
192 
193             projectList=list(leftover)+optionals
194 
195             #Incase an individual does not group every student
```

```
196             if len(projectList)<1:
197                 individual.fitness=-1
198                 break
199 
200             project=random.choice(projectList)
201 
202             #If the project does not yet have a group, add minSize number
    ↪  of students
203             if project in optionals:
204                 newProject=Project(project.ID,project.name,project.skills
    ↪ ,project.maxSize,project.minSize,project.optional)
205                 optionals.remove(project)
206                 temp=list(students)
207                 random.shuffle(temp)
208                 group=temp[:project.minSize]
209                 students-=set(group)
210                 newProject.addStudent(group)
211                 individual.addProject(newProject)
212 
213                 #If project is not full, allow for it to get more
    ↪ students
214                 if project.minSize!=project.maxSize:
215                     leftover[project.ID]=project.maxSize-project.minSize
216 
217             #If project already had a group, add one more student to it
218             else:
219                 projectID=project
220                 student=random.choice(list(students))
221                 individual.projects[projectID].addStudent(student)
222                 students.remove(student)
223 
224                 #Remove from dict if now full
225                 leftover[projectID]-=1
226                 if leftover[projectID]<=0:
227                     del leftover[projectID]
228 
229 
230         return individual
231 
232     def mutate(self):
233         return self
234 
235     def __str__(self):
236         string=""
237         for i in self.projects:
238             string+= self.projects[i].__str__()+"\n"
239         return string
240 
241     #Returns the names, size, gender count, and skill count for each
    ↪ project as a dict with the projects' IDs as keys
```

```
242     def get_data(self):
243         data=dict()
244         for project in self.projects:
245             project=self.projects[project]
246 
247             genders,skills,students=project.get_data()
248             projectData={"name":project.name,"students":students,"size":
    ↪ len(project.group),"genders":genders,"skills":skills}
249 
250             data[project.ID]=projectData
251 
252         return data
253 
254 
255 
256 #=====SUBROUTINES=====
257 #Fetch student details from database
258 def get_students(database,IDs):
259      students=[]
260 
261      with sql.connect(database) as conn:
262           cur = conn.cursor()
263 
264           for ID in IDs:
265                #Get each students' skills
266                cur.execute("SELECT Skill FROM StudentSkills WHERE
    ↪ StudentID = ?",(ID,))
267                skills=set()
268                result=cur.fetchall()
269                for skill in result:
270                     skills.add(skill[0])
271 
272                #Get each students' name and gender
273                cur.execute("SELECT Name, Gender from Students WHERE
    ↪ StudentID = ?",(ID,))
274                entry=cur.fetchall()[0]
275 
276                #Create a Student object for each student with their name,
    ↪  skills, and gender
277                students.append(Student(ID,entry[0],skills,entry[1]))
278 
279      return students
280 
281 
282 #Fetch project details from database
283 def get_projects(database,IDs):
284      projects={"Optional":[],"notOptional":[]}
285 
286      with sql.connect(database) as conn:
287           cur = conn.cursor()
```

```
288
289          for ID in IDs:
290               #Get each projects' skill requirements
291               cur.execute("SELECT Skill FROM ProjectSkills WHERE
     ↪ ProjectID = ?",(ID,))
292               skills=set()
293               result=cur.fetchall()
294               for skill in result:
295                    skills.add(skill[0])
296
297               #Get each projects' data other than skills
298               cur.execute("SELECT Name, MaxSize, MinSize, Optional from
     ↪ Projects WHERE ProjectID = ?",(ID,))
299               entry=cur.fetchall()[0]
300
301               #Create a Project object for each project with its name,
     ↪ skills, maxSize, minSize, and whether or not it is optional
302               if bool(entry[3]):
303                    projects["Optional"].append(Project(ID,entry[0],
     ↪ skills,entry[1],entry[2],entry[3]))
304               else:
305                    projects["notOptional"].append(Project(ID,entry[0],
     ↪ skills,entry[1],entry[2],entry[3]))
306
307
308     return projects
309
310 #find optional projects whose minSize is not too large to house the
     ↪ remaining ungrouped students
311 def get_optionals(optionals,extraSlots):
312     optionals.sort(key=lambda x:x.minSize, reverse=True)
313     for i in range(len(optionals)):
314          if optionals[i].minSize>extraSlots:
315               optionals.pop(i)
316          else:
317               break
318     return optionals
319
320
321 #Creates an individual for the first population
322 def GenInitialGroups(projects,students):
323     individual=Individual()
324     leftover=[] #Stores projects that have groups, but do not have the
     ↪ maximum number of students assigned yet
325
326     #Fill every required project to minSize with random students
327     required=projects["notOptional"]
328     for i in range(len(required)):
329          required[i]=required[i].deepcopy()
330     for project in required:
```

```
331             random.shuffle(students)
332             project.addStudent(students[:project.minSize])
333             individual.addProject(project)
334 
335             #Remove grouped students
336             students=students[project.minSize:]
337 
338             #Keep projects with room to spare so that they may take on more
    ↪  students later
339             if project.minSize!=project.maxSize:
340                 leftover.append(project.ID)
341 
342         #Ensure every student is grouped
343         optionals=[] #Stores projects that do not yet have a group assigned
344         for i in range(len(projects["Optional"])):
345             optionals.append(projects["Optional"][i].deepcopy())
346 
347         while len(students)>0:
348             #Only allow students to be grouped to a project with a minSize
    ↪ that allows the remaining students to be housed
349             optionals=get_optionals(optionals,len(students))
350 
351             #Incase an individual does not group every student
352             if len(leftover+optionals)<1:
353                 individual.fitness=-1
354                 break
355             #Add random students to a random project
356             project=random.choice(leftover+optionals)
357             random.shuffle(students)
358 
359             #If the project has no group currently, add minSize number of
    ↪ students
360             if type(project)==Project:
361                 optionals.remove(project)
362                 project.addStudent(students[:project.minSize])
363                 individual.addProject(project)
364 
365                 students=students[project.minSize:]
366 
367                 #If project is not full, allow for it to gain more
    ↪ students
368                 if project.minSize!=project.maxSize:
369                     leftover.append(project.ID)
370 
371             #If project already has a group, add one student to it
372             else:
373                 individual.projects[project].addStudent(students[0])
374                 students.pop(0)
375 
376                 #Remove from list if now full
```

```
377                 if len(individual.projects[project].group)>=individual.
    ↪ projects[project].maxSize:
378                     leftover.remove(project)
379      return individual
380
381
382
383
384
385 #Runs the genetic algorithm
386 def run(database, studentIDs, projectIDs, TARGET_FITNESS, MAX_ITERATIONS,
    ↪  POPULATION_SIZE):
387      #List of students
388      students=get_students(database,studentIDs)
389      #Two lists of projects in dictionary, separated by whether or not
    ↪ the project is optional
390      projects=get_projects(database,projectIDs)
391
392      #Ensure there aren't too many or too few students to meet
    ↪ requirements#
393      numStudents=len(students)
394      minStudents=0
395      maxStudents=0
396
397      #Find max and min amount of students required
398      for project in projects["notOptional"]:
399           maxStudents += project.maxSize
400           minStudents += project.minSize
401      for project in projects["Optional"]:
402           maxStudents += project.maxSize
403      #Return error for too few
404      if numStudents<minStudents:
405           return {"status":-1,"error":f"Error: Only {len(students)} to
    ↪ fill a minimum of {minStudents} places."}
406      #Return error for too many
407      elif numStudents>maxStudents:
408           return {"status":-1,"error":f"Error: {len(students)} is too
    ↪ many to fit in {maxStudents} places."}
409
410      #Generate initial groupings and average fitness
411      population = []
412      for i in range(POPULATION_SIZE):
413           individual = GenInitialGroups(projects, students)
414           individual.calcFitness()
415           population.append(individual)
416
417      #=====Genetic algorithm======
418      for i in range(MAX_ITERATIONS):
419           population.sort(key=lambda x:x.fitness,reverse=True)
420           #Ends early when an individual has reached the target fitness
```

```
            if population[0].fitness>=TARGET_FITNESS:
                return {"status":0,"results":population[0].get_data(),"all
    ↪ ":population}

            #Eliminate individuals in the bottom half in terms of fitness
            population = population[:POPULATION_SIZE//2]
            new_pop = []

            #Create next population
            crossedSize=int(0.9*POPULATION_SIZE)
            for j in range(crossedSize):
                #Crossover random individuals
                parents=random.sample(population,2)
                individual=parents[0].crossover(parents[1],students)

                #Random mutations to prevent convergence towards a local
    ↪ best solution
                individual=individual.mutate()
                individual.calcFitness()

                new_pop.append(individual)

            #Create the next population with the crossover population and
    ↪ the best 10% of the last population
            population=population[:(POPULATION_SIZE-crossedSize)]
            population+=new_pop
        population.sort(key=lambda x:x.fitness,reverse=True)
        return {"status":0,"results":population[0].get_data(),"all":
    ↪ population}



#=====TESTING======
if __name__=="__main__":
    result=run("Code/database.db",[1,3,4,5,6],[4,5,6],0.9,5,50)
    if result["status"]==0:
        pop=result["all"]
        for ind in pop:
            print(ind.fitness)
            print(ind)
```

3.3 CreateTables.sql

```
-- SQLite
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS Students (
```

```
    StudentID INTEGER PRIMARY KEY,
    Name TEXT,
    Gender TEXT
);

CREATE TABLE IF NOT EXISTS StudentSkills (
    StudentID INTEGER NOT NULL,
    Skill TEXT NOT NULL,
  PRIMARY KEY (StudentID, Skill),
  FOREIGN KEY (StudentID) REFERENCES Students (StudentID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Projects (
    ProjectID INTEGER PRIMARY KEY,
    Name TEXT,
    MaxSize INTEGER,
    MinSize INTEGER,
    Optional INTEGER
);

CREATE TABLE IF NOT EXISTS ProjectSkills (
    ProjectID INTEGER NOT NULL,
    Skill TEXT NOT NULL,
  PRIMARY KEY (ProjectID, Skill),
  FOREIGN KEY (ProjectID) REFERENCES Projects (ProjectID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Results (
    ResultID INTEGER PRIMARY KEY,
    ProjectID INTEGER NOT NULL,
    LocalTimeGenerated TEXT,
    EpochTimeGenerated INTEGER,
    FOREIGN KEY (ProjectID) REFERENCES Projects (ProjectID)
        ON DELETE CASCADE
        ON UPDATE CASCADE

);

CREATE TABLE IF NOT EXISTS ResultStudentLink (
    ResultID INTEGER NOT NULL,
    StudentID INTEGER NOT NULL,
  PRIMARY KEY (ResultID, StudentID),
    FOREIGN KEY (ResultID) REFERENCES Results (ResultID)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
  FOREIGN KEY (StudentID) REFERENCES Students (StudentID)
```

```
55          ON DELETE CASCADE
56          ON UPDATE CASCADE
57  );
```

4 Testing

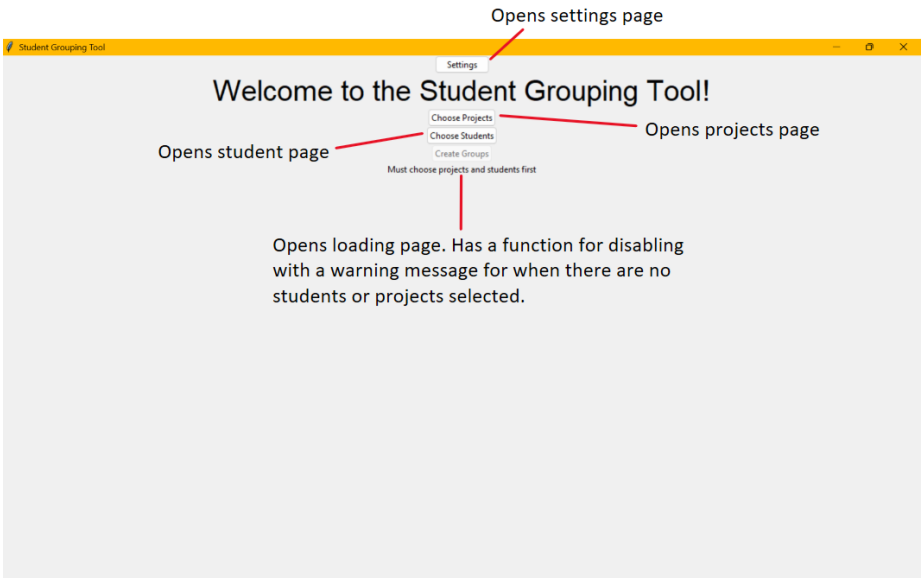


Figure 21: The home page

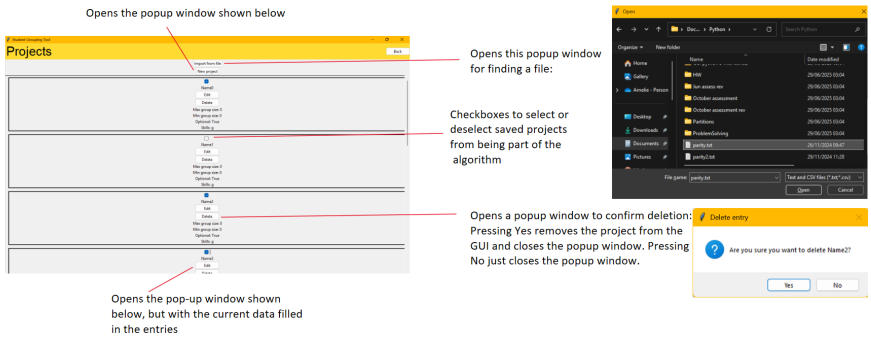


Figure 22: The project page

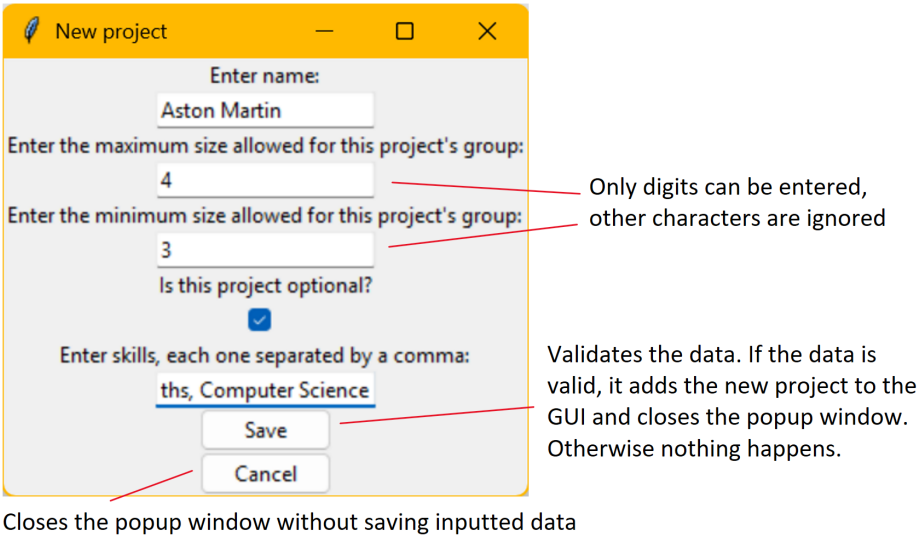


Figure 23: The popup window for entering data about a new project

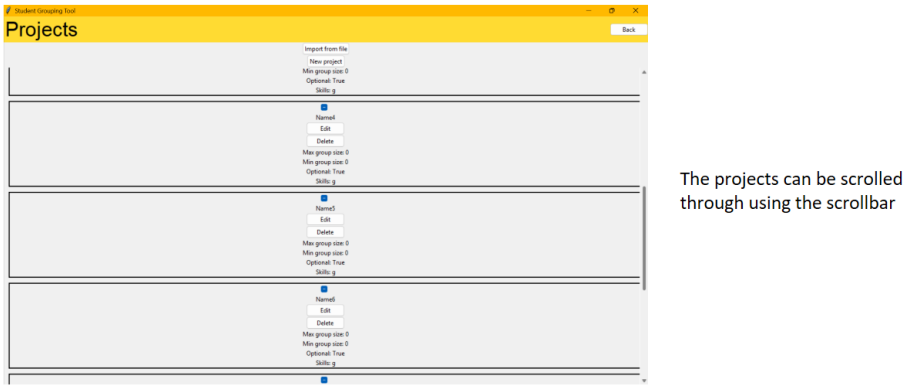


Figure 24: The project page, scrolled down

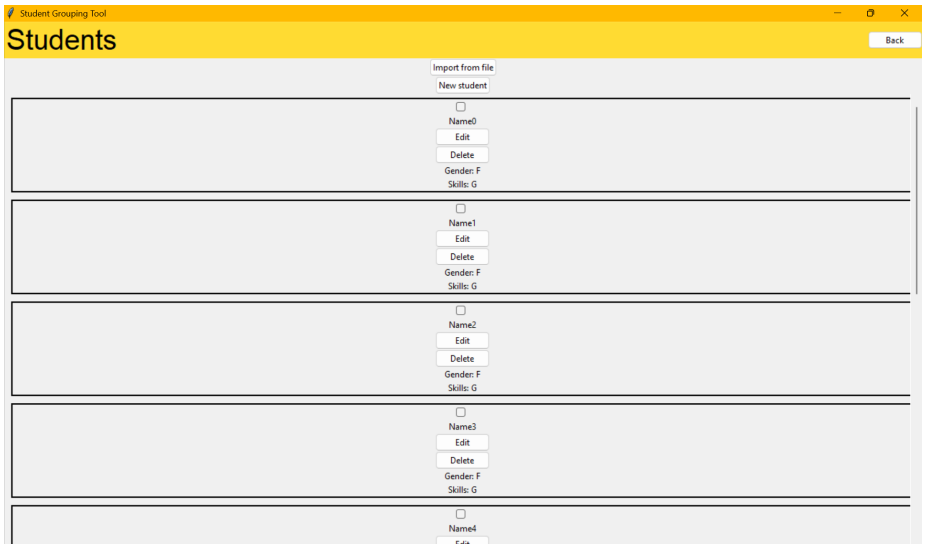


Figure 25: The student page. Functions identically to the projects page

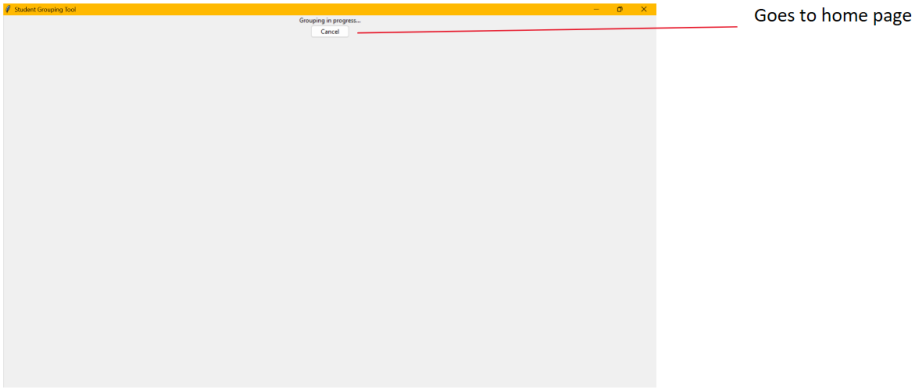


Figure 26: The loading page. It runs the genetic algorithm on page load and opens the results page when results have been returned.

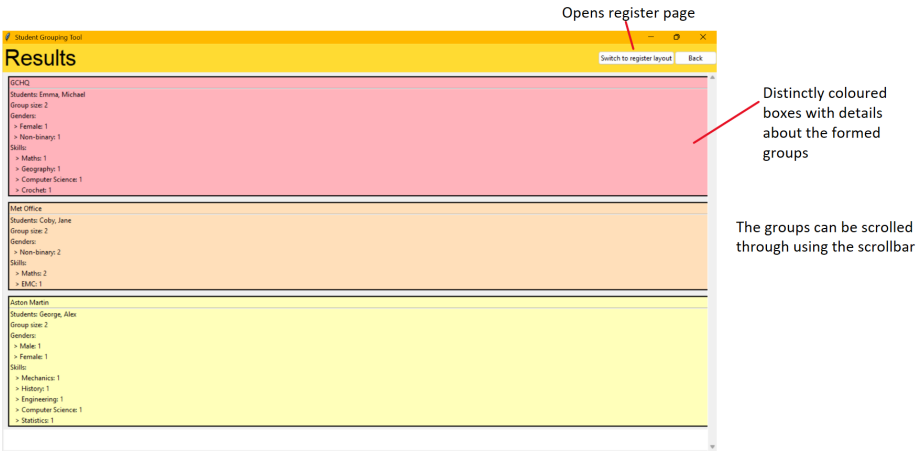


Figure 27: The results page

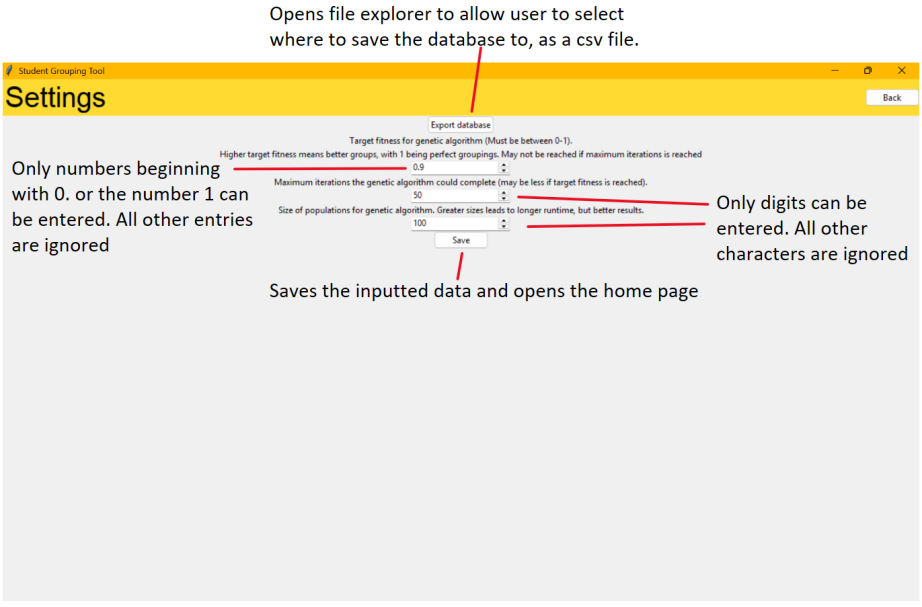


Figure 28: The settings page

4.1 Video

4.2 Testing plan

ID	Obj.	Description	Test Data	Expected Results	Time stamp
1	1a,1e	Check that all student fields are required when entered manually	Name: Tina Gender: Female Skills: Physics, Chemistry 1.1,1.2,1.3 (Erroneous): Name, gender, or skills box is blank 1.4 (Normal): All data above is entered	1.1,1.2,1.3: A warning message displays to the user, the pop-up window remains open, and the data does not get saved. 1.4: The pop-up window closes, the data is displayed on the GUI and saved to the database.	
2	1a,1e,4a	Check that the entered student data is saved when all fields are completed correctly	(Normal): Same as 1.4	An SQL SELECT statement returns the newly added data.	
3	1h, 4a	Check that student data can be edited	(Normal): Edit Tina Name: Dylan Gender: Male Skills: Chemistry, Maths, Biology	An SQL SELECT statement only shows the new data, none of the old data.	
4	1h, 4a	Check that student data can be deleted	(Normal): Delete Dylan	An SQL SELECT statement shows that no data about Dylan exists	
5	1c, 1d	Check that student data is saved from previous uses of the program	(Normal): Close and re-open the program	The data shown on the student page is exactly the same as how it was left	
6	1bi, 1e, 4a	Check that program only accepts CSVs for student data in the required format	6.1 (Erroneous): 6.2 (Normal):	6.1: A warning message displays and the data does not get saved. 6.2: The data is displayed on the GUI and saved to the database.	

ID	Obj.	Description	Test Data	Expected Results	Time stamp
8	1a,1fi	Check that the project's maximum group size must be greater than or equal to minimum group size when entered manually	8.1 (Erroneous): 8.2 (Boundary): 8.3 (Normal):	8.1: A warning message displays to the user, the pop-up window remains open, and the data does not get saved. 8.2: The pop-up window closes, the data is displayed on the GUI and saved to the database. 8.3: The pop-up window closes, the data is displayed on the GUI and saved to the database.	
9	1a,1fii	Check that the project's minimum group size must be greater than 0 when entered manually			
10	1a,1f	Check that all project fields are required			
11	1a,1f,4a	Check that the entered project data is saved when all fields are completed correctly			
12	1h,4a	Check that project data can be edited			
13	1h,4a	Check that project data can be deleted			
14	1c	Check that project data is saved from previous uses of the program			

15	1bi,1d	Check that project data can be imported through csv files when all fields are correct			
16	1bi,1fi	Check that the project's maximum group size must be greater than minimum group size when importing a csv file			
17	1bi,1fii	Check that the project's minimum group size must be greater than 0 when importing a csv file			
18	1bi,1f	Check that all project fields are required when importing a csv file			
19	1gi	Check that setting's target fitness must be between 0 and 1			
20	1gii	Check that setting's max iterations must be a positive integer, not including 0			
21	1giii	Check that setting's size of populations must be a positive integer, not including 0			
22	4bi	Check that database can be exported as a csv file			

23	4c	Check that settings data is stored as a JSON			
24	1i	Check that at least one student, one project, target fitness, max iterations, and size of populations must all be entered before the algorithm is run			

5 Evaluation

5.1 Objective evaluation

The program successfully takes in student and project data, validates it, runs a grouping algorithm, and outputs the results onto a GUI.

[Testing showed that this objective was achieved/not achieved for multiple objectives]

5.2 Third-party feedback

1.

5.3 Feedback evaluation

5.4 Improvements

[Optional objectives] [More input data] [Exporting outputs]